# Improving Security and Performance of Financial RESTful APIs Using ChatGPT in ASP.NET Core

By

**ZAW YE HTUT KO**

Advisor: Assistant Professor Dr. Darun Kesrarat

Master Thesis Full Report

**Submitted in Partial Fulfilment of the Requirements for the Degree of Master of Science in Information Technology, Assumption University**

March 2026

Vincent Mary School of Engineering, Science and Technology

Thesis Full Report Approval

| | |
|---|---|
| Thesis Title | Improving Security and Performance of Financial RESTful APIs Using ChatGPT in ASP.NET Core |
| By | ZAW YE HTUT KO |
| Thesis Advisor | ASST PROF. DR. DARUN KESRARAT |
| Academic Year | 2/2025 |

The Vincent Mary School of Engineering, Science and Technology of Assumption University has approved this full thesis report as partial fulfilment of the requirements for the degree of Master of Science in Information Technology.

Approval Committee:

| | |
|---|---|
| (……) | (……) |
| Advisor | Committee Member |
| (……) | (……) |
| Committee Member | Program Director |

_____/_____

Month /Year

# Abstract

This full report evaluates how ChatGPT (Codex 5.4), used as an AI-assisted code improvement advisor, can improve the security, performance, and code quality of a financial accounting RESTful API implemented in ASP.NET Core. The study uses one existing API codebase, not a public financial dataset, and measures before-and-after changes across three independent tool gates: SonarQube for static code quality, OWASP ZAP for security scanning, and Apache JMeter for load performance. The evaluated API includes authentication, users, accounts, journal entries, accounting periods, financial reports, and audit logs. Baseline measurements are taken from the baseline-v0.1 branch, while post-improvement measurements are taken from dedicated branches for each tool: baseline-sonarqube-v1, baseline-zap-v1, and baseline-jmeter-v1. The results show that the SonarQube branch reduced retained C# findings from 28 to 0 and achieved an OK quality gate; the ZAP branch cleared all gated High, Medium, and Low business-endpoint alerts; and the JMeter branch produced large p95 latency reductions across normal, soak, and spike profiles, with the remaining caveat that the mixed-workload drift heuristic still fails. The report therefore demonstrates that ChatGPT can be useful as an advisor when its recommendations are applied on isolated branches and verified by repeatable tools, while also showing that human review and automated regression testing remain necessary. The conference reviews requested clearer representative code evidence, explicit prompt templates, stronger result interpretation, and clearer limits on generalizability and human intervention. The revised final report therefore keeps the existing branch evidence and adds prompt-template and human-effort traceability so the report aligns with the camera-ready manuscript.

---

# Table of Contents

#### 4.5.4 Overall Effectiveness Assessment

## List of Acronyms

AI - Artificial Intelligence
API - Application Programming Interface
ASP.NET - Active Server Pages .NET
BAL - Business Access Layer
CI/CD - Continuous Integration / Continuous Delivery
COEP - Cross-Origin Embedder Policy
COOP - Cross-Origin Opener Policy
CORP - Cross-Origin Resource Policy
CORS - Cross-Origin Resource Sharing
CPU - Central Processing Unit
CRUD - Create, Read, Update, Delete
CSRF - Cross-Site Request Forgery
CSP - Content Security Policy
CWE - Common Weakness Enumeration
DAST - Dynamic Application Security Testing
DTO - Data Transfer Object
EF - Entity Framework
GPT - Generative Pre-trained Transformer
GUID - Globally Unique Identifier
HTTP - Hypertext Transfer Protocol
IDE - Integrated Development Environment
IEEE - Institute of Electrical and Electronics Engineers
JSON - JavaScript Object Notation
JWT - JSON Web Token
LLM - Large Language Model
MVC - Model-View-Controller
NER - Named Entity Recognition
NLP - Natural Language Processing
OWASP - Open Worldwide Application Security Project
RBAC - Role-Based Access Control
REST - Representational State Transfer
SDK - Software Development Kit
SLA - Service-Level Agreement
SQL - Structured Query Language
TSV - Tab-Separated Values
UI - User Interface

URL - Uniform Resource Locator
VU - Virtual User
XSS - Cross-Site Scripting
ZAP - Zed Attack Proxy

# Chapter 1 - Introduction

## 1.1 Introduction

RESTful APIs are web interfaces that expose resources through standard HTTP methods such as GET, POST, PUT, and DELETE. They are widely used in financial systems because they let client applications, reporting tools, and back-office services exchange account, journal-entry, period, report, and audit-log data through a predictable contract. Prior Web API research emphasizes that API reliability, documentation, security, and performance are central adoption concerns (Zhang et al., 2022). In a financial accounting context, weak authentication, unsafe input handling, slow reporting endpoints, or unclear access-control rules can directly affect confidentiality, integrity, and system availability.

ChatGPT (Codex 5.4), accessed through OpenAI's Codex coding environment, is used in this study as an advisory tool rather than as an autonomous developer. The model receives scanner output, error traces, and code context, then suggests code changes. A human developer reviews those suggestions, applies selected changes on isolated Git branches, and verifies them with the same tool that produced the baseline finding. This distinction matters because prior studies show that LLM-generated code can introduce security weaknesses if it is accepted without review (Pearce et al., 2022; Khoury et al., 2023; Sandoval et al., 2023).

**Why this work matters.** Production REST APIs in regulated domains such as finance fail in three distinct ways: a static-analysis violation (a code smell that creeps in over time), a security alert (a missing security header, a vulnerable bundled dependency, an unprotected route), and a performance hotspot (a response that breaks an SLA under load). The three failures are caught by three different tools, demand three different remediation styles, and are usually owned by three different teams. If a general-purpose LLM such as ChatGPT can give correct, framework-aware advice across all three at once, using only the structured tool report and the relevant code excerpt, it materially shortens the audit-to-fix cycle that is the bottleneck of modern API quality work. Establishing whether and where this works requires the kind of branch-isolated empirical evidence that this thesis proposes to collect, and the answer is directly actionable for practitioners deciding whether to embed LLMs in their CI/CD review loop.

**Expected outcomes summary.** On the financial-accounting ASP.NET Core API used as the unit of analysis, the study expects ChatGPT (Codex 5.4) acting purely as an advisor on tool output to (i) substantially reduce SonarQube findings while preserving test coverage and quality-gate status; (ii) clear gated High, Medium, and Low alerts in OWASP ZAP baseline and authenticated API scans on business endpoints; and (iii) achieve double-digit percentage reductions in p95 latency under JMeter load profiles at 50, 100, and 500 virtual users while keeping error rate at 0%. The study further expects to surface at least one honest limitation: structural metrics that depend on global workload composition, rather than on per-endpoint latency, are not expected to

move under per-finding LLM patches. Section 1.5.2 elaborates the per-dimension expected outcomes in concrete numerical terms.

---

### 1.2 Challenges

APIs are crucial for modern web and mobile applications because they allow independent systems to exchange data and trigger business operations. In finance, this includes authentication, account management, journal-entry posting, financial reporting, and audit-log retrieval. Because these operations handle sensitive records, API failures can expose user identity data, financial balances, and operational audit trails. Security and performance therefore need to be evaluated together: an API that is secure but too slow can fail operational requirements, while a fast API with weak access control can create unacceptable risk.

APIs that handle sensitive data are frequent targets for attack. SQL injection occurs when untrusted input is allowed to alter a database query; Cross-Site Scripting (XSS) occurs when unsafe content can be executed by a client; and Cross-Site Request Forgery (CSRF) abuses an authenticated user session to submit unintended actions. RESTful APIs also require strong authentication, role-based authorization, input validation, secure headers, and careful error handling. In this study, OWASP ZAP is used to detect web/API security weaknesses, while role-based tests verify that Admin, FinanceManager, User, and Auditor permissions are enforced correctly.

---

### 1.3 Problem Statement

The problem addressed in this report is that traditional API improvement work is fragmented. Static analysis identifies maintainability issues, security scanners identify missing headers or vulnerability patterns, and load tests identify slow endpoints, but these findings are often handled separately. For the financial accounting API used in this study, the current challenge is to determine whether ChatGPT (Codex 5.4) can convert those tool findings into practical code improvements without introducing regressions. The report therefore compares baseline-v0.1 with three isolated post-improvement branches and asks whether the same tools show measurable gains after the ChatGPT-guided changes.

---

### 1.4 Motivational Statement

The motivation behind this research is practical. Manual API hardening is time-consuming because a developer must read scanner output, identify the affected code path, understand the security or performance implication, implement a fix, run regression tests, and repeat the measurement. This loop is especially slow when the issue crosses layers such as controller authorization, service logic, Entity Framework queries, response headers, and test setup. ChatGPT may reduce this time by helping interpret tool findings and proposing fix candidates, but its output must still be validated because incorrect advice can create new security, performance, or maintainability problems.

---

1.5 Objectives, Measurements, and Outcomes

#### 1.5.1 Objectives:

- To evaluate the impact of ChatGPT on the quality of ASP.NET Core API projects.
- To assess the performance impacts of AI-generated code through speed tests.
- To measure improvements in code security, reliability, and maintainability using **SonarQube**.
- To demonstrate how ChatGPT can solve security problems like **authentication** and **authorization** in RESTful APIs using AI-generated guidelines.

1.5.2 Measured Outcomes:

- A side-by-side before-and-after comparison of the API project following ChatGPT-guided code recommendations, using branch names, tool outputs, and measurement units for each comparison.
- Code quality evidence from SonarQube, including C# finding count, Bugs, Vulnerabilities, Code Smells, Security Hotspots, coverage percentage, duplicated-line density percentage, and Quality Gate status.
- Security evidence from OWASP ZAP, including High, Medium, Low, and Informational alert counts before and after improvements, plus role-based access-control checks for Admin, FinanceManager, User, and Auditor.
- Performance evidence from Apache JMeter, including average response time, p95 and p99 latency in milliseconds, throughput in transactions per second, error rate percentage, and soak/spike drift percentage.

### 1.6 Limitations

The scope of this full report is limited to one ASP.NET Core financial accounting RESTful API and to ChatGPT (Codex 5.4) accessed through OpenAI's Codex coding environment. The study does not compare multiple AI assistants. The analysis focuses on automated static analysis, security scanning, and load testing; it does not evaluate user experience or developer productivity through a human-subject study. The findings therefore support conclusions about this API and this measurement workflow, not universal claims about all APIs or all LLMs.

## Chapter 2 - Literature Review

Recent systematic reviews of LLMs in software engineering (Hou et al., 2024) show that existing research is dominated by code-generation studies, while the use of LLMs as advisors acting on reports from static analysers, security scanners, and load-testing tools is comparatively under-explored. This positions the present study within an under-represented quadrant of the LLM-for-SE literature.

### 2.1. The Impact of AI-generated Code on Web Development A Comparative Study of ChatGPT and GitHub Copilot

**Introduction to the Literature Review**

- Focuses on using ChatGPT and GitHub Copilot to generate complete websites.
- Reviews machine learning advancements in code generation.
- Highlights the limited prior studies on comparing AI tools for complete web applications involving multiple programming languages.

**Key Themes or Topics in the Literature**

- Generative AI models and their application in code synthesis.
- Key aspects of code quality: ease of use, accuracy, and maintainability.
- Ethical implications and societal impact of AI in web development.

**Key Findings from the Literature**

- Both tools can generate functional websites, though ChatGPT was rated as more user-friendly.
- The generated code had minor discrepancies in styling and functionality.
- AI tools are not yet advanced enough to fully replace developers for complex projects.

**Gaps in the Literature**

- Insufficient exploration of AI performance in multi-language systems.

- Limited research on long-term code maintenance and security.

**Methodologies and Tools in the Literature**

- Static analysis tools like SonarQube, ESLint, and Pylint for code evaluation.
- Experiments involved recreating wireframes for comparison.

**Critical Analysis**

- The study underscores the potential of AI tools but also highlights their current limitations, such as bug generation and security risks.

Reason to Include: This paper compares ChatGPT and GitHub Copilot in web development, focusing on the accuracy, ease of use, and maintainability of AI-generated code. The study's use of SonarQube to assess code quality provides insights into how ChatGPT can be applied to improve code security and performance, making it relevant to this research.

---

### 2.2. Methodology for Code Synthesis Evaluation of LLMs Presented by a Case Study of ChatGPT and Copilot

**Introduction to the Literature Review**

- Explores the methodology for evaluating large language models (LLMs) like ChatGPT and Copilot.
- Discusses the significance of understanding how developers choose tools for code generation.

**Key Themes or Topics in the Literature**

- Functional validity and technical quality in code synthesis.
- Impact of prompt engineering on LLM performance.
- Role of static analysis in measuring non-functional qualities like code security.

**Key Findings from the Literature**

- ChatGPT performed better on average, generating more correct solutions than Copilot.
- Both tools displayed recurring code smells and vulnerabilities.
- Prompt engineering emerged as a critical factor influencing code quality.

**Gaps in the Literature**

- Lack of research on real-world deployment challenges for AI-generated code.
- Limited studies exploring how complex tasks affect AI-generated code accuracy.

**Methodologies and Tools in the Literature**

- Benchmarked against 25 programming tasks using C++ and Java.
- Combined human review and static analysis for quality assessment.

**Critical Analysis**

- Provides a robust methodology for evaluating LLM-generated code but suggests the need for dynamic testing and deployment-specific evaluations.

Reason to Include: This paper explores how ChatGPT improves software development by generating code, detecting bugs, and automating tests. The focus on how ChatGPT enhances traditional development processes supports the research's aim to optimize API development using AI.

---

### 2.3. Artificial Intelligence-Based Tools

**Introduction to the Literature Review**

- Highlights the evolving role of AI in automating traditional software development processes.
- Reviews how tools like ChatGPT enhance workflows across design, coding, and testing.

**Key Themes or Topics in the Literature**

- Efficiency improvements through AI in software development cycles.
- The potential for AI tools to accelerate error detection and debugging.
- Ethical concerns, such as over-reliance on AI and security implications.

**Key Findings from the Literature**

- ChatGPT was effective in automating repetitive tasks and reducing development time.
- Simplified processes for non-specialist developers to engage in software projects.

**Gaps in the Literature**

- Limited case studies on how AI integrates with advanced frameworks.
- Lack of large-scale evaluations across diverse industries.

**Methodologies and Tools in the Literature**

- AI-driven processes were benchmarked against traditional methodologies.
- Observations from simulated projects and user feedback.

**Critical Analysis**

- While demonstrating efficiency, the study underlines the risks of over-dependency on AI for critical functions, emphasizing the need for human oversight.

Reason to Include: This paper explores how ChatGPT improves software development by generating code, detecting bugs, and automating tests. The focus on how ChatGPT enhances traditional development processes supports the research's aim to optimize API development using AI.

---

### 2.4. GPTutor

**Introduction to the Literature Review**

- Examines GPTutor, a plugin leveraging ChatGPT for code explanation and debugging.
- Highlights its focus on student and developer education.

**Key Themes or Topics in the Literature**

- Enhancing learning with AI-driven code explanation tools.
- Integration of natural language processing (NLP) into programming environments.
- Comparison of GPTutor's effectiveness with existing tools like Copilot.

**Key Findings from the Literature**

- GPTutor provided concise and context-aware explanations of code.
- Developers found it intuitive and useful for understanding unfamiliar codebases.

**Gaps in the Literature**

- Lack of real-world deployment data to validate GPTutor's performance in professional settings.
- Minimal studies on its integration with other educational platforms.

**Methodologies and Tools in the Literature**

- Implementation as a Visual Studio Code extension using OpenAI's GPT-3.5 API.
- User interviews and experimental evaluations.

**Critical Analysis**

- Shows promise in improving developer education but requires further studies to explore its long-term applicability.

Reason to Include: This paper is highly relevant to this research on using ChatGPT to generate and improve code, particularly in the context of explaining programming logic. GPTutor leverages ChatGPT to assist developers with real-time code explanations, which aligns with your objective of improving API development using ChatGPT for better security and performance.

---

### 2.5. Web APIs Features, Issues, and Expectations – A Large-Scale Empirical Study of Web

**Introduction to the Literature Review**

- Analyzes features, usability issues, and user expectations for web APIs based on empirical studies.

**Key Themes or Topics in the Literature**

- Factors influencing API adoption, such as documentation and SDK quality.
- Issues like outdated components and incomplete documentation.
- Developers' expectations for better error handling and performance optimization.

**Key Findings from the Literature**

- Well-organized documentation is critical for API adoption.
- Security issues and limited functionality were common concerns.

**Gaps in the Literature**

- Limited exploration of API usage in large-scale applications.
- Minimal studies on machine-learning-driven API enhancements.

**Methodologies and Tools in the Literature**

- Survey responses from developers and analysis of Stack Overflow discussions.
- Evaluation of APIs from registries like Programmable Web.

**Critical Analysis**

- Demonstrates the importance of improving API usability but lacks insights into ML applications for automated API management.

Reason to Include: This empirical study provides a large-scale analysis of common issues and expectations in Web APIs, including security vulnerabilities, performance bottlenecks, and developer expectations. These findings directly support this research on addressing similar API challenges using ChatGPT-generated improvements.

---

### 2.6 An Investigation into Misuse of Java Security APIs by Large Language Models

**Introduction to the Literature Review**

- Evaluates how LLMs like ChatGPT handle Java security APIs, focusing on misuse risks.

**Key Themes or Topics in the Literature**

- Common misuse patterns in security APIs.
- ChatGPT's limitations in implementing secure code due to its reliance on outdated examples.

**Key Findings from the Literature**

- Around 70% of ChatGPT-generated solutions had security API misuse.
- Developers need to provide highly specific prompts to mitigate misuse.

**Gaps in the Literature**

- Limited studies on real-world usage of AI tools for secure coding.
- Absence of standardized datasets for API security evaluation.

**Methodologies and Tools in the Literature**

- Manual review and CryptoGuard for identifying misuse in 48 Java tasks.

**Critical Analysis**

- Highlights the potential of AI in secure development but stresses the importance of training on updated datasets.

**Reason to Include:** This paper assesses ChatGPT's reliability in generating secure code for Java security APIs, highlighting common misuse patterns. Its findings on API misuse and limitations in automated detection align closely with my research goal of improving ASP.NET Core API security with ChatGPT-guided improvements, offering valuable insights into potential vulnerabilities and evaluation methods.

---

### 2.7 A New Approach to Web Application Security Utilizing GPT Language Models for Source Code Inspection

**Introduction to the Literature Review**

- Focuses on using GPT language models for static code analysis to detect vulnerabilities, specifically targeting CWE-653 (improper isolation of sensitive data).

**Key Themes or Topics in the Literature**

- Use of GPT models for identifying security vulnerabilities in web frameworks like Angular.
- Integration of AI for automating sensitive data protection and isolation checks.
- Challenges in using GPT models for context-specific vulnerability detection.

**Key Findings from the Literature**

- GPT models achieved an 88.76% detection success rate for CWE-653 vulnerabilities.
- Static code analysis combined with few-shot learning and chain-of-thought techniques enhanced detection accuracy.
- The models proved effective in mapping sensitive data paths and categorizing protection levels.

**Gaps in the Literature**

- Insufficient real-world applications of the proposed methodology.
- Limited comparative studies on the performance of GPT against other advanced vulnerability detection tools.

**Methodologies and Tools in the Literature**

- Analysis of open-source Angular applications using static code inspection.
- Categorization of sensitive data based on damage potential from leaks.
- Comparison of GPT-3.5 and GPT-4 for detecting and addressing vulnerabilities.

**Critical Analysis**

- The study presents a compelling case for using GPT models in vulnerability detection, but it underscores the necessity of augmenting GPT outputs with human expertise for broader security scenarios.

Reason to Include: This paper explores using GPT models for security inspection in web applications, like this study focuses on ChatGPT for enhancing RESTful API security. Its findings on detecting vulnerabilities like CWE-653 and categorizing sensitive data in web applications align well with research, providing insights into leveraging GPT models for automated code review, vulnerability detection, and the practical challenges of AI-assisted security improvements. This context helps justify the value of ChatGPT in identifying security flaws in ASP.NET Core APIs within this research.

---

### 2.8. Evaluating the Code Quality of AI-Assisted Code Generation Tools

**Introduction to the Literature Review**

- Compares GitHub Copilot, Amazon CodeWhisperer, and ChatGPT to assess their performance in generating reliable, secure, and maintainable code.

**Key Themes or Topics in the Literature**

- Analysis of AI-generated code quality metrics: correctness, security, maintainability, and reliability.
- Exploration of developer productivity and technical debt reduction using AI tools.
- Effects of docstrings and function signatures on AI-generated code quality.

**Key Findings from the Literature**

- ChatGPT excelled in correctness (65.2%) and maintainability, while Copilot had better IDE integration.
- Amazon CodeWhisperer performed best for tasks involving cloud service APIs.
- The average technical debt was lowest for Amazon CodeWhisperer but ChatGPT's explanations were more user-friendly.

**Gaps in the Literature**

- Absence of comprehensive studies involving multi-language or highly complex codebases.
- Limited exploration of adaptive learning and feedback loops in AI systems.

**Methodologies and Tools in the Literature**

- Utilized the HumanEval dataset for benchmarking.
- Static analysis was performed using SonarQube for bugs, code smells, and security vulnerabilities.

**Critical Analysis**

- While AI tools show promise, the findings highlight challenges in scalability and real-world deployment, calling for further research in enhancing AI's adaptive learning.

Reason to Include: This paper evaluates the code quality of AI-assisted generation tools, including ChatGPT, focusing on essential metrics such as code validity, correctness, security, reliability, and maintainability. Since my research involves using ChatGPT to improve security and performance in ASP.NET Core RESTful APIs, this study provides a valuable framework for assessing ChatGPT's code quality. The paper's empirical approach and use of SonarQube align with my methodology, offering insights into common issues with AI-generated code and suggesting best practices for achieving secure, maintainable, and efficient code with AI assistance.

---

### 2.9 Evaluation of Common Security Vulnerabilities of State Universities and Colleges Websites Based on OWASP

**Introduction to the Literature Review**

- Analyzes the security vulnerabilities in academic institution websites using OWASP’s Top 10 guidelines, emphasizing the importance of web application security in digitalized educational services.

**Key Themes or Topics in the Literature**

- Frequent vulnerabilities in web applications, including SQL injection, misconfiguration, and broken access control.
- Using OWASP ZAP to identify and analyze security gaps.
- Importance of adopting OWASP standards to enhance website security.

**Key Findings from the Literature**

- 94.12% of websites were vulnerable to broken access control, and 88.24% showed security misconfigurations.
- Vulnerabilities often stem from outdated components and poor system design.
- ZAP proved effective in identifying critical issues, including injection flaws and insecure designs.

**Gaps in the Literature**

- Lack of studies on integrating AI-based tools to complement traditional scanners like ZAP.
- Minimal exploration of user training for maintaining secure web applications.

**Methodologies and Tools in the Literature**

- Automated scanning of 17 academic websites using OWASP ZAP.
- Qualitative analysis of scan results based on the OWASP Top 10 categories.

**Critical Analysis**

- Demonstrates the utility of OWASP ZAP but underlines the need for proactive measures and advanced AI-driven solutions to mitigate vulnerabilities.

Reason to Include: this paper examines real-world security vulnerabilities in web systems using OWASP ZAP, aligning closely with this study focuses on securing RESTful APIs. The study provides insights into common security flaws (e.g., injection vulnerabilities, broken access control) in academic web platforms, which can parallel the types of vulnerabilities found in RESTful APIs. Additionally, their methodology using OWASP's Top 10 framework offers a structured approach to vulnerability assessment that I can apply to evaluate ChatGPT-driven security improvements in my APIs. This context supports the need for systematic, tool-assisted security evaluation in API development.

---

### 2.10 The Impact of AI on Developer Productivity Evidence from GitHub Copilot

**Introduction to the Literature Review**

- Evaluates GitHub Copilot's impact on developer productivity through a controlled experiment involving a task to build an HTTP server.

**Key Themes or Topics in the Literature**

- Productivity enhancements from AI pair programming.
- Heterogeneous effects of Copilot across skill levels and demographics.
- Comparison of traditional development workflows versus AI-assisted programming.

**Key Findings from the Literature**

- Developers using Copilot completed tasks 55.8% faster on average than those in the control group.
- The tool benefitted less experienced developers the most, showcasing its potential for supporting novice programmers.
- Participants rated Copilot as significantly improving their productivity and willingness to adopt it in daily workflows.

**Gaps in the Literature**

- Limited insights into the long-term adoption effects of AI tools on collaborative and large-scale software projects.
- Lack of focus on the ethical and licensing implications of using AI for code generation.

**Methodologies and Tools in the Literature**

- Controlled experiment using GitHub Classroom to track task completion times and correctness.
- Analysis of heterogeneous effects based on participant demographics and experience.

**Critical Analysis**

- Highlights Copilot's potential to accelerate software development but emphasizes the need for addressing ethical concerns and improving support for team-based projects.

Reason to Include: This paper is relevant to my research because it provides empirical evidence on the productivity gains achieved through AI-assisted coding tools, specifically GitHub Copilot, which is like ChatGPT in functionality. By highlighting the productivity boost and its impact on code development speed, the study offers a comparison point for evaluating ChatGPT's potential in improving API performance and security, supporting your investigation into the effectiveness of AI-driven development tools.

### 2.11 Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions

**Introduction to the Literature Review**

Audits GitHub Copilot's code suggestions against security-relevant scenarios drawn from MITRE's list of the 25 most dangerous Common Weakness Enumerations (CWEs), focusing on whether AI-generated code introduces exploitable vulnerabilities rather than on functional correctness alone.

**Key Themes or Topics in the Literature**

Security risks of AI pair-programming tools in everyday development workflows.

Mapping of AI-generated vulnerabilities to standardised CWE categories.

Trade-offs between generation diversity, prompt context, and security outcomes.

**Key Findings from the Literature**

Roughly 40% of Copilot's generated programs across the evaluation scenarios contained security weaknesses.

Copilot's output varied by language and weakness class, with C programs exhibiting a higher proportion of vulnerabilities than Python.

Prompt wording and surrounding context materially influenced whether Copilot produced vulnerable or safe code.

**Gaps in the Literature**

Restricted to Copilot at a single point in time and to two languages (Python and C); behaviour on ASP.NET Core / C# is not characterised.

No empirical link between flagged weaknesses and downstream exploitability in running applications.

**Methodologies and Tools in the Literature**

89 scenarios covering diversity of weaknesses, prompts, and application domains.

CodeQL and manual inspection used to triage generated code against CWE criteria.

**Critical Analysis**

Provides one of the earliest systematic security audits of an AI code assistant and establishes that AI suggestions must be treated as untrusted input to a security-review process, not as trusted code.

**Reason to Include:** This paper directly motivates why the present study layers an independent security scanner (OWASP ZAP) on top of ChatGPT-generated recommendations. If AI-assistant output is empirically vulnerable in roughly 40% of cases on Copilot, a symmetric security-scan gate is necessary when evaluating ChatGPT-guided improvements on an ASP.NET Core financial API.

### 2.12 How Secure is Code Generated by ChatGPT?

**Introduction to the Literature Review**

Evaluates the security properties of code produced by ChatGPT across several programming languages and realistic security-relevant tasks, extending the Copilot-centric security audits of Section 2.11 to the specific model family used in this thesis.

**Key Themes or Topics in the Literature**

Security-aware prompt engineering versus default prompting.

ChatGPT's awareness of vulnerabilities it has itself produced.

Effect of follow-up prompts in driving ChatGPT to repair unsafe code.

**Key Findings from the Literature**

Only a small minority of the generated programs were secure on the first attempt across the evaluated tasks.

When explicitly challenged, ChatGPT was often capable of recognising and remediating the weakness, but did not proactively produce secure code by default.

Security quality depended strongly on language and on whether the prompt explicitly asked for secure code.

**Gaps in the Literature**

Evaluation targets generation tasks and does not assess ChatGPT acting as an advisor on an already-existing code base.

Limited coverage of web-framework-specific vulnerability classes such as missing security headers, broken RBAC, and JWT handling.

**Methodologies and Tools in the Literature**

Prompt-driven generation across 21 tasks in five languages.

Manual security inspection of each generated output.

**Critical Analysis**

Provides direct evidence that unfiltered ChatGPT output cannot be assumed secure, while also showing that iterative dialog with the model can materially raise the security bar. This is a key assumption behind the advisor-loop methodology used in this study.

**Reason to Include:** The present study uses ChatGPT (Codex 5.4) specifically as a remediation advisor on scanner findings. Khoury et al.'s finding that ChatGPT improves under follow-up prompting supports the four-phase workflow in Chapter 3, in which ZAP and SonarQube results are fed back to ChatGPT as structured prompts rather than open-ended requests to write secure code.

### 2.13 Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants

**Introduction to the Literature Review**

Conducts a controlled user study on whether access to an LLM-based code assistant causes developers to introduce more security vulnerabilities into their code than they would without one, complementing tool-only evaluations (Sections 2.11 and 2.12) with a human-factors perspective.

**Key Themes or Topics in the Literature**

Participant-introduced vulnerabilities under LLM assistance.

Effect of AI suggestions on developer confidence and code-review effort.

Interaction between developer experience level and AI-assistant outcomes.

**Key Findings from the Literature**

Access to an LLM code assistant did not produce a statistically significant increase in the number of security bugs relative to the unassisted control group in the tasks studied.

Participants sometimes accepted AI suggestions uncritically but also rejected insecure suggestions, so the net security effect was small in this study design.

The finding contrasts with simpler narratives that AI-generated code is inherently insecure, and highlights the role of the human in the loop.

**Gaps in the Literature**

Focuses on short, well-scoped C programming tasks rather than on maintaining a non-trivial production code base.

Does not evaluate how developers interact with scanner output and an LLM assistant simultaneously, which is the specific workflow used in this thesis.

**Methodologies and Tools in the Literature**

Between-subjects user study with a programming task and AI assistant access as the treatment.

Manual code-review coding of introduced vulnerabilities across participant submissions.

**Critical Analysis**

Indicates that LLM assistants are neither an automatic security win nor an automatic security loss. Outcomes depend on task, developer, and review process, which supports using LLMs as advisors gated by automated measurement tools rather than as autonomous agents.

**Reason to Include:** This paper justifies the methodological choice in Chapter 3 to use ChatGPT only as an advisor whose recommendations are applied on an isolated Git branch and re-validated by the same scanner that surfaced the finding. This arrangement directly addresses the risk that a developer may accept an insecure AI suggestion, as flagged by Sandoval et al.

### 2.14 Large Language Models for Software Engineering: A Systematic Literature Review

**Introduction to the Literature Review**

Provides a broad Systematic Literature Review of the use of Large Language Models across the software-engineering life-cycle, synthesising a large corpus of primary studies spanning requirements, design, code generation, testing, and maintenance.

**Key Themes or Topics in the Literature**

Taxonomy of LLM applications in software engineering by task, covering generation, review, repair, documentation, and testing.

Model-family coverage, prompt-engineering practices, and dataset curation strategies.

Reported strengths, weaknesses, and open research questions across the SE tasks surveyed.

**Key Findings from the Literature**

The literature is dominated by code-generation studies, while code review, repair, and quality-advisory use cases are comparatively under-represented.

Evaluation methodologies are heterogeneous, which makes cross-study comparison difficult.

Security, performance, and maintainability are often studied in isolation rather than jointly.

**Gaps in the Literature**

Few primary studies evaluate LLMs as improvement advisors on an existing code base under combined static-analysis, security-scan, and load-test gates.

Limited empirical work uses enterprise-style REST APIs as the unit of analysis.

**Methodologies and Tools in the Literature**

Standard SLR protocol covering search-string design across major digital libraries, inclusion and exclusion criteria, quality assessment, and thematic synthesis.

**Critical Analysis**

Positions the present thesis within the broader LLM-for-SE landscape and substantiates the claim in Chapter 1 that the existing literature emphasises generation quality while this study focuses on advisory quality on a fixed, pre-existing API.

**Reason to Include:** This study's combination of an existing non-trivial API, three independent quality dimensions measured with standard tools, and an LLM-as-advisor framing addresses an empirically under-represented quadrant of the LLM-for-SE research space identified by Hou et al.

### 2.15 Are SonarQube Rules Inducing Bugs?

**Introduction to the Literature Review**

Empirically investigates whether violations flagged by SonarQube rules are actually associated with bug-inducing code changes in real software projects, directly interrogating the validity of SonarQube-derived quality metrics as proxies for true software quality.

**Key Themes or Topics in the Literature**

Distinction between rule violations and bug-inducing changes in version-controlled projects.

Statistical association between SonarQube severity levels and defect rates.

Implications for using static-analysis counts as quality-improvement evidence.

**Key Findings from the Literature**

Only a minority of SonarQube rules show a statistically significant relationship with induced bugs, while many rules flag issues with little observed defect impact.

Severity classifications assigned by SonarQube do not consistently align with empirical defect-inducing behaviour.

Interpreting fewer SonarQube violations as fewer real bugs is therefore only partly warranted without corroborating evidence.

**Gaps in the Literature**

Analysis is restricted to a sample of open-source Apache projects; generalisation to proprietary enterprise code bases is not demonstrated.

Does not evaluate SonarQube in combination with independent security scanners or load-test results, which is precisely the multi-instrument setting used in this thesis.

**Methodologies and Tools in the Literature**

SZZ-style bug-linking analysis on 21 Apache projects.

SonarQube scans combined with statistical correlation of rule violations to bug-inducing commits.

**Critical Analysis**

Provides an important methodological caution: absolute SonarQube counts cannot, on their own, establish that code quality has improved. Cross-validation against independent tools such as dynamic scanners and load tests materially strengthens quality claims.

**Reason to Include:** This paper motivates the deliberately multi-instrument design in Chapters 3 and 4. Reductions in SonarQube findings on the baseline-sonarqube-v1 branch are interpreted alongside ZAP security outcomes (baseline-zap-v1) and JMeter performance results (baseline-jmeter-v1), explicitly acknowledging the Lenarduzzi et al. critique that SonarQube counts alone are a limited proxy for real-world quality.

## Chapter 3: Research Overview and Methodology

What this full report covers

This full report focuses on improving the security, performance, and code quality of one ASP.NET Core financial accounting RESTful API by using ChatGPT (Codex 5.4) as a code-improvement advisor. The API surface evaluated in the report includes authentication, user profile access, chart of accounts, journal entries, accounting periods, financial reports, and audit logs. The study compares baseline-v0.1 with three isolated improvement branches: baseline-sonarqube-v1 for code quality, baseline-zap-v1 for security, and baseline-jmeter-v1 for performance. The toolchain consists of SonarQube, OWASP ZAP, Apache JMeter, dotnet test, and Git branch-based comparison.

---

**Why is it important?**

This work is important because RESTful APIs are critical components in modern software systems, yet they often suffer from vulnerabilities such as SQL injection, missing security headers, weak authorization, and performance bottlenecks under load. Securing and optimizing these APIs is essential for protecting sensitive data and maintaining reliable service. This report demonstrates how ChatGPT can assist developers by interpreting tool findings and suggesting fixes, while the final acceptance decision remains based on repeatable measurements rather than trust in AI output alone.

**Chapter overview.** This chapter is organized into six subsections that together describe how the study will be carried out. Section 3.1 outlines the four-phase research workflow (baseline, ChatGPT-guided improvement, post-improvement testing, comparative analysis) and the per-tool branch isolation that ties every measured delta to a single tool. Section 3.2 describes the dataset used as the test bed for security and performance evaluation. Section 3.3 lists the three measurement tools (SonarQube, Apache JMeter, OWASP ZAP) and the AI assistant (ChatGPT, Codex 5.4) that the study selects. Section 3.4 describes how ChatGPT-guided improvements are implemented on each branch. Section 3.5 details the testing phases applied before and after the improvements. Section 3.6 contrasts this study with related work along two axes: a single-API focus and a multi-tool combined evaluation framework.

---

**Figure (3.0) Four-phase workflow for baseline capture and branch-isolated remediation evidence.**

| Phase | Purpose | Evidence branch |
|---|---|---|
| 1. Baseline capture | Run the v0.1 branch against SonarQube, OWASP ZAP, and JMeter before remediation. | baseline-v0.1 |
| 2. Static-code remediation | Apply maintainability and code-smell fixes, then preserve rule IDs and affected files for traceability. | baseline-sonarqube-v1 |
| 3. Security remediation | Move security headers ahead of Swagger/OpenAPI, update vulnerable Swagger dependencies, and reduce exposed UI surface. | baseline-zap-v1 |
| 4. Performance remediation | Optimize report generation, account-ledger access, database indexes, caching, and compression, then rerun JMeter profiles. | baseline-jmeter-v1 |

This chapter is organized as an evidence-based remediation workflow rather than a tool checklist. The project first records a measurable baseline, then isolates static-analysis, security, and load-test fixes on separate branches so each improvement can be compared against the same starting point.

## Testing Environment and Reproducibility

| Environment item | Value | Why it matters |
|---|---|---|
| API runtime | ASP.NET Core 8.0 Web API in Development mode | Application under test |
| Database | SQL Server local database named Financial | Repeatable local test data |
| Host | Windows 11 workstation | Same host used for baseline and fix-branch |

| | | |
|---|---|---|
| | | runs |
| Load test tool | Docker image justb4/jmeter:5.5 | JMeter p50, p100, p500, soak, spike profiles |
| Security test tool | Docker image ghcr.io/zaproxy/zaproxy:stable | OWASP ZAP baseline and authenticated API scans |
| Branches compared | baseline-v0.1, baseline-sonarqube-v1, baseline-zap-v1, baseline-jmeter-v1 | Controls before/after evidence |

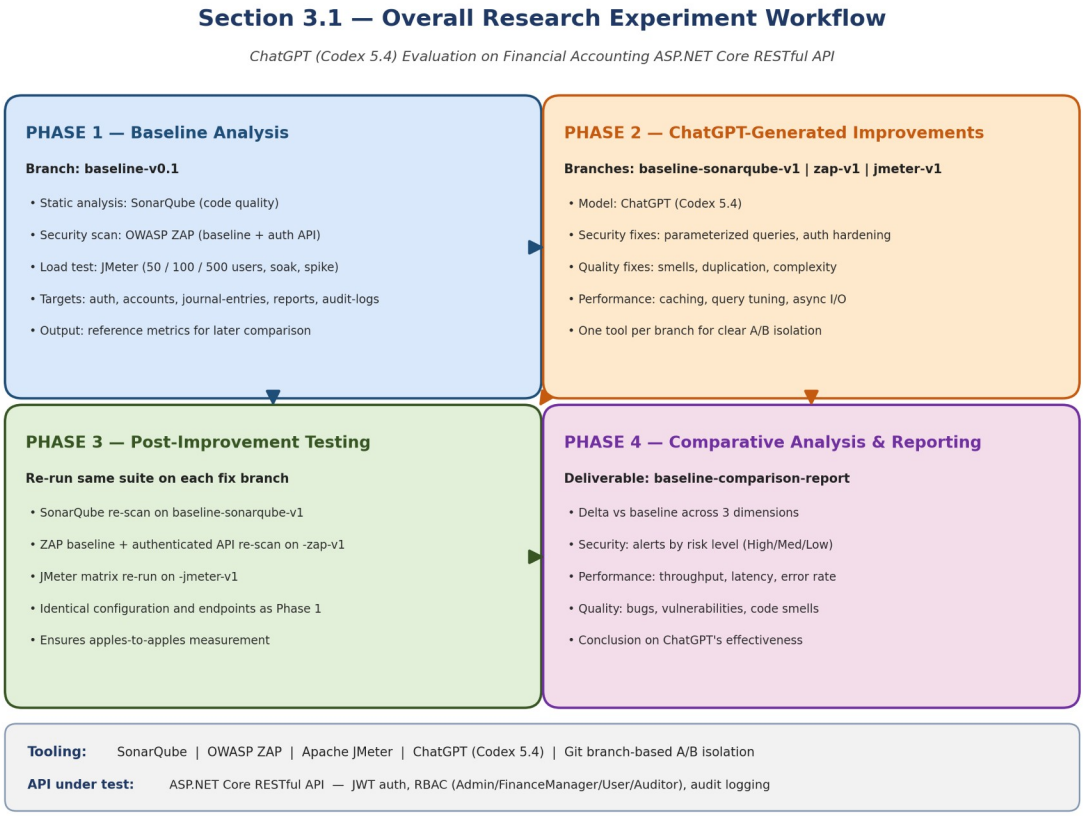
All comparisons were executed against the same ASP.NET Core 8.0 financial-accounting API and the local SQL Server Financial database on a Windows 11 host. This matters because the performance claims depend on keeping the runtime, database, and host constant while changing only the remediation branch.

JMeter was run through the Docker image justb4/jmeter:5.5, and OWASP ZAP was run through the stable container image ghcr.io/zaproxy/zaproxy:stable. The branch sequence used for comparison was baseline-v0.1, baseline-sonarqube-v1, baseline-zap-v1, and baseline-jmeter-v1.

The seeded dataset used 120 accounts, 30,000 journal entries, and at least 5,000 posted entries for report-generation paths. The primary authenticated roles were Admin, FinanceManager, User, and Auditor, which allowed the tests to exercise both normal accounting workflows and protected administrative surfaces.

API scope used in the study: the report evaluates seven endpoint groups. Authentication uses POST /auth/login. User identity uses GET /users/me and admin user creation through POST /users. Accounts include GET /accounts, GET /accounts/{accountId}, GET /accounts/{accountId}/balance, POST /accounts, and PUT /accounts/{accountId}. Journal entries include POST /journal-entries, POST /journal-entries/bulk, GET /journal-entries, GET /journal-entries/{id}, POST /journal-entries/{id}/post, POST /journal-entries/{id}/reverse, and DELETE /journal-entries/{id}. Periods include GET /periods, POST /periods, and POST /periods/{periodId}/close. Reports include GET /reports/trial-balance, GET /reports/profit-loss, GET /reports/balance-sheet, and GET /reports/account-ledger. Audit logs use GET /audit-logs. These endpoints are enough for the study because they cover authentication, authorization, CRUD operations, bulk write operations, financial reporting, and audit traceability.

Issue categories used for comparison: security issues are grouped by OWASP ZAP risk level and vulnerability class such as missing security headers, vulnerable JavaScript library exposure, CORS/header hardening, SQL injection exposure, authentication behavior, and access-control behavior. Performance issues are grouped by endpoint and load profile, using average latency, p95/p99 latency in milliseconds, throughput in transactions per second, error rate percentage, soak drift, and spike recovery drift. Code-quality issues are grouped by SonarQube category, including Bugs, Vulnerabilities, Code Smells, Security Hotspots, Quality Gate status, coverage percentage, duplicated-line density, and specific rule IDs such as S125, S6966, S107, and S1192.




Workflow details for the four-phase research method.

- **Phase 1: Baseline Analysis**
    - Initial security and performance testing are conducted on the financial accounting ASP.NET Core RESTful API project. The API includes endpoints for authentication, chart of accounts, journal entries, accounting periods, financial reports, and audit logs. These baseline metrics are captured on the baseline-v0.1 branch and provide reference points for evaluating the effectiveness of improvements.
    - Tools: SonarQube for code quality and static analysis, OWASP ZAP for security vulnerability scanning (baseline scan and authenticated API scan), and Apache JMeter for performance testing (load, soak, and spike test profiles at 50, 100, and 500 concurrent users).

**Example:**

- **Financial Accounting API: A complex RESTful API for financial accounting with multi-tier architecture (API, BAL/Services, MODEL/Entities, Tests).**
    - **Example Endpoints: GET /accounts to list chart of accounts, POST /journal-entries to create journal entries, GET /reports/trial-balance to generate trial balance reports.**

- **The API features role-based access control (Admin, FinanceManager, User, Auditor), JWT authentication, and comprehensive audit logging.**
    - **Example Endpoint: POST /journal-entries/bulk to create bulk journal entries with debit/credit line validation, requiring authenticated access and finance role authorization.**

- **Phase 2: ChatGPT-Generated Improvements**
    - Code recommendations are generated using ChatGPT (Codex 5.4) through OpenAI's Codex coding environment. ChatGPT receives tool findings, relevant code snippets, and expected constraints, then suggests candidate fixes. The candidate fixes are not accepted automatically: each one is reviewed, applied on a dedicated branch, tested with dotnet test, and re-measured with the same tool that produced the baseline finding. This process explains why a recommendation is considered useful only when the after-measurement improves without breaking regression tests.
    - Example Guidance from ChatGPT: Using parameterized queries to prevent SQL injection, improving loop efficiency, implementing secure session management.

**Example:**

**Security Enhancement:**

- Baseline Issue: Security and validation risk in write endpoints such as POST /journal-entries and POST /journal-entries/bulk, where untrusted input must not affect database query behavior or bypass authorization.
- ChatGPT Recommendation: preserve Entity Framework parameterization, validate DTO input, reject unauthorized roles, and keep write endpoints protected by JWT authentication and role policies.

**Figure (3.1) API endpoint coverage used for side-by-side security and performance evidence.**

| Category | Representative endpoints | Evidence source | Reviewer relevance |
|---|---|---|---|
| Authentication | /api/auth/register, /api/auth/login, /api/auth/me | ZAP authenticated scan, API smoke coverage | Role-aware access and token-protected flows |
| Users | /api/users, /api/users/{id}, /api/users/{id}/status | ZAP authenticated scan | Administrative user management |
| Accounts | /api/accounts, /api/accounts/{id}, /api/accounts/{id}/ledger | JMeter core and complex profiles | Ledger browsing and account-level reporting |
| Journal entries | /api/journalentries, /api/journalentries/{id}, /api/journalentries/post | JMeter bulk and query profiles | High-volume transaction workflow |
| Periods | /api/periods, /api/periods/{periodId}/close | API smoke coverage | Accounting period setup and close controls |
| Reports | /api/reports/trial-balance, /api/reports/profit-loss, /api/reports/balance-sheet | JMeter complex, soak, and spike profiles | Performance-critical financial reporting |
| Audit logs | /api/auditlogs | ZAP authenticated scan and manual role checks | Traceability for sensitive accounting actions |

**Performance Optimization:**

- Baseline Issue: Slow p95 latency in report endpoints, especially GET /reports/trial-balance, GET /reports/profit-loss, GET /reports/balance-sheet, and GET /reports/account-ledger during spike and soak profiles.
- **ChatGPT Recommendation:** Implement caching for frequently accessed endpoints.
    - **Example Adjustment: Use in-memory caching for the GET /reports endpoint to store results for 10 minutes:**

**Figure (3.2) SonarQube code-quality remediation evidence by rule, file, and fix.**

| Rule or area | File | Before | After | Status |
|---|---|---|---|---|
| S107 | JournalEntriesController.cs | Controller action used eight query parameters. | Introduced JournalEntryQueryDto and passed one query model to the service. | Code smell removed |
| S2325 | ServiceManager.cs | Private helper did not access instance state. | Converted helper to static. | Code smell removed |
| S3267/S6602 | JournalEntryService.cs | Loops and LINQ operations were less maintainable. | Rewrote selected loops and lookup calls with clearer collection operations. | Code smell removed |
| S6966 | Program.cs | Application startup used blocking Run. | Changed startup to await app.RunAsync(). | Code smell removed |
| Summary | C# project | 28 retained SonarQube findings in baseline. | 0 retained findings after fixes; Quality Gate remained OK. | Closed |

- **Phase 3: Post-Improvement Testing**
    - With ChatGPT-guided changes implemented on each branch, the project undergoes a new round of testing using the same tools and configurations as the baseline. Security, performance, and code quality metrics are collected again and compared to the baseline-v0.1 results.

    **Example:**

    - **Security: Run ZAP baseline scan and authenticated API scan against all endpoints (auth, accounts, journal-entries, periods, reports, audit-logs) on the baseline-zap-v1 branch.**
    - **Performance: Run JMeter test matrix (50, 100, 500 concurrent users + soak + spike profiles) against all API endpoints on the baseline-jmeter-v1 branch.**

- **Phase 4: Comparative Analysis and Reporting**
    - The data collected from the baseline (baseline-v0.1) and post-improvement branches (baseline-sonarqube-v1, baseline-zap-v1, baseline-jmeter-v1) are analyzed and compared to draw conclusions on ChatGPT's effectiveness in enhancing the security, performance, and code quality of the financial accounting RESTful API. Results are compiled into a baseline-comparison-report documenting improvements across all three dimensions.

**Figure (3.3) OWASP ZAP before-and-after security evidence.**

| Scan | Severity | Before -> After | Vulnerability class | Fix applied |
|---|---|---|---|---|
| Baseline scan | Medium | 2 -> 0 | Missing CSP and exposed Swagger UI behavior. | SecurityHeadersMiddleware moved before Swagger; Swagger UI disabled unless explicitly enabled. |
| Baseline scan | Low | 6 -> 0 | Missing X-Content-Type-Options and modern isolation headers. | Added nosniff, COEP, COOP, CORP, and Permissions-Policy headers. |
| Baseline scan | Informational | 3 -> 4 | Residual informational notices after hardening. | Accepted as non-blocking because high/medium/low alerts were cleared. |
| Authenticated API scan | Low | 3 -> 0 | Authenticated endpoint header findings. | Same header middleware path now protects authenticated API responses. |
| Dependency risk | Low/Informational | DOMPurify warning removed | Swagger UI dependency bundled vulnerable DOMPurify. | Upgraded Swashbuckle.AspNetCore and Swagger filters packages. |

**Figure (3.4) JMeter before-and-after performance profile comparison.**

| Profile | Metric | Baseline | After fixes | Change | Status |
|---|---|---|---|---|---|
| p50 | Total p95 response time | 49.45 ms | 13.00 ms | -73.71% | PASS |
| p100 | Total p95 response time | 261.95 ms | 34.00 ms | -87.02% | PASS |
| p500 | Total p95 response time | 176.00 ms | 28.00 ms | -84.09% | PASS |
| p100 | Throughput | 57.53 req/s | 59.82 req/s | +3.98% | PASS |
| p500 | Error rate | 0.00% | 0.00% | No regression | PASS |
| Soak | Total p95 response time | 970.95 ms | 56.00 ms | Improved | Drift still monitored |
| Spike | Total p95 response time | 6795.00 ms | 1420.00 ms | Improved | Recovery drift still monitored |

**Figure (3.5) Selected before-and-after code evidence linked to reviewer comments.**

| Example | Before branch | Before code | After branch | After code | Reason |
|---|---|---|---|---|---|
| SonarQube maintainability | baseline-v0.1 | public async Task<IActionResult> GetPaged(DateTime? from, DateTime? to, string? status, int? accountId, string? search, int page = 1, int pageSize = 20, string? sort = "entryDate_desc") | baseline-sonarqube-v1 | public async Task<IActionResult> GetPaged([FromQuery] JournalEntryQueryDto query) | Replaced eight query parameters with one DTO, directly addressing S107. |
| ZAP security headers | baseline-v0.1 | app.UseSwagger(); app.UseSwaggerUI(); ... app.UseMiddleware<SecurityHeadersMiddleware>(); | baseline-zap-v1 | app.UseMiddleware<SecurityHeadersMiddleware>(); ... if (enableSwaggerDocument) app.UseSwagger(); if (enableSwaggerUi) app.UseSwaggerUI(); | Ensures Swagger/OpenAPI responses receive the same hardening headers and separates the UI switch from the JSON document. |
| JMeter report performance | baseline-v0.1 | await _context.Reports.AddAsync(report); await _context.SaveChangesAsync(); | baseline-jmeter-v1 | await _auditLogService.WriteAsync(actorUserId, "GENERATE_REPORT", "Reports", $"{reportType}:{periodId}", ipAddress, new { reportType, periodId, items = items.Count }); | Removed report snapshot write amplification from read-heavy report generation. |

## Selected Source-Code Evidence

The following excerpts are real source-code evidence from the compared Git branches. They are intentionally selected examples rather than full-file listings, because the purpose is to show how the measured SonarQube, ZAP, and JMeter improvements map back to concrete code changes.

### Source-code evidence A: SonarQube maintainability fix in API/Controllers/JournalEntriesController.cs.

### Before: baseline-v0.1

```
[HttpGet]
public async Task<IActionResult> GetPaged(
    [FromQuery] DateTime? from,
    [FromQuery] DateTime? to,
    [FromQuery] string? status,
```

```
    [FromQuery] int? accountId,
    [FromQuery] string? search,
    [FromQuery] int page = 1,
    [FromQuery] int pageSize = 20,
    [FromQuery] string? sort = "entryDate_desc")
{
    var result = await _journalEntryService.GetPagedAsync(from, to, status, accountId, search, page,
pageSize, sort);
    return Ok(result);
}
```

### After: baseline-sonarqube-v1

```
[HttpGet]
public async Task<IActionResult> GetPaged([FromQuery] JournalEntryQueryDto query)
{
    var result = await _journalEntryService.GetPagedAsync(query);
    return Ok(result);
}
```

This excerpt addresses the SonarQube maintainability issue by replacing a long query-parameter list with a single JournalEntryQueryDto request model.

### Source-code evidence B: OWASP ZAP security-header fix in API/Program.cs.

### Before: baseline-v0.1

```
var enableSwagger = app.Environment.IsDevelopment() || appSettings.EnableSwaggerInProduction;
if (enableSwagger)
{
    app.UseSwagger();
    app.UseSwaggerUI();
}


app.UseMiddleware<CorrelationIdMiddleware>();
app.UseMiddleware<GlobalExceptionMiddleware>();
app.UseMiddleware<SecurityHeadersMiddleware>();
```

### After: baseline-zap-v1

```
app.UseMiddleware<CorrelationIdMiddleware>();
app.UseMiddleware<GlobalExceptionMiddleware>();
app.UseMiddleware<SecurityHeadersMiddleware>();


var enableSwaggerDocument = app.Environment.IsDevelopment() || appSettings.EnableSwaggerInProduction;
var enableSwaggerUi = enableSwaggerDocument && appSettings.EnableSwaggerUi;
if (enableSwaggerDocument)
{
    app.UseSwagger();
}


if (enableSwaggerUi)
{
    app.UseSwaggerUI();
}
```

This excerpt shows that the security-header middleware was moved before Swagger/OpenAPI middleware, so diagnostic responses receive the same hardening headers as API responses.

### Source-code evidence C: JMeter performance fix in BAL/Services/FinancialReportService.cs.

### Before: baseline-v0.1

```
private async Task PersistReportAsync(
    string reportType,
```

```
    int periodId,
    Guid actorUserId,
    string? ipAddress,
    List<ReportItemDto> items)
{
    var report = new FinancialReport
    {
        ReportType = reportType,
        PeriodId = periodId,
        GeneratedByUserId = actorUserId,
        GeneratedAt = DateTime.UtcNow
    };

    foreach (var item in items)
    {
        report.ReportItems.Add(new FinancialReportItem
        {
            AccountId = item.AccountId,
            DebitTotal = decimal.Round(item.DebitTotal, 2),
            CreditTotal = decimal.Round(item.CreditTotal, 2),
            Balance = decimal.Round(item.Balance, 2)
        });
    }

    await _context.Reports.AddAsync(report);
    await _context.SaveChangesAsync();
}
```

### After: baseline-jmeter-v1

```
public async Task<AccountLedgerPayload> GetAccountLedgerPayloadAsync(int accountId, int periodId, Guid
actorUserId, string? ipAddress)
{
    var period = await GetPeriodOrThrowAsync(periodId);
    var account = await _context.ChartOfAccounts.AsNoTracking().FirstOrDefaultAsync(x => x.AccountId ==
accountId);

    var payload = await _accountLedgerCache.GetOrCreatePayloadAsync(
        accountId,
        periodId,
        account.AccountType,
        period.StartDate,
        period.EndDate,
        () => BuildAccountLedgerPayloadAsync(accountId, period, account));

    await PersistReportAsync("Ledger", periodId, actorUserId, ipAddress, new List<ReportItemDto> { ... });
    return payload;
}

private async Task PersistReportAsync(string reportType, int periodId, Guid actorUserId, string? ipAddress,
List<ReportItemDto> items)
{
    await _auditLogService.WriteAsync(
        actorUserId,
        "GENERATE_REPORT",
        "Reports",
        $"{reportType}:{periodId}",
        ipAddress,
        new { reportType, periodId, items = items.Count });
}
```

This excerpt shows the core performance change: report generation no longer persists full report snapshots during read-heavy paths, and account-ledger output can be served from the new cache-backed payload path while preserving audit logging.

## Prompt-template evidence added for camera-ready reproducibility.

The remediation prompts used the same three-part structure across SonarQube, OWASP ZAP, and JMeter findings. This makes the LLM interaction auditable: the tool report states the measurable problem, the code excerpt bounds the editable context, and the instruction constrains the model to a minimal patch that must be re-tested.

| Prompt component | Content used in this study | Purpose |
|---|---|---|
| Context | Native tool finding: SonarQube rule id, ZAP alert metadata, or JMeter endpoint latency profile. | Anchors the model to a measured defect or hotspot. |
| Code excerpt | Smallest affected controller, service, middleware, configuration, or test block. | Prevents broad rewrites and keeps the proposed patch reviewable. |
| Instruction | Propose a minimal patch that keeps existing tests green, avoids new dependencies unless justified, and states how to re-measure the result. | Turns the LLM output into an auditable repair proposal rather than an accepted change. |

## Human-effort traceability.

The following table records how much human intervention was required before a candidate fix was accepted. The important point is that ChatGPT supplied advice, but the acceptance gate was the developer review plus the same measurement tool that produced the original finding.

| Fix class | Evidence branch | LLM rounds | Human intervention | Acceptance evidence |
|---|---|---|---|---|
| SonarQube controller DTO refactor | baseline-sonarqube-v1 | 1 | Reviewed route compatibility and service-layer signature. | Retained C# findings reduced from 28 to 0. |
| Security headers and Swagger hardening | baseline-zap-v1 | 1-2 | Moved middleware order, split Swagger document/UI switches, reviewed package upgrade. | ZAP High/Medium/Low gated alerts cleared. |
| ARM/camelCase analyzer cleanup | baseline-sonarqube-v1 | 2-3 | Manually corrected provider-specific conventions after initial model output was insufficient. | Analyzer accepted final template text. |
| Report-performance remediation | baseline-jmeter-v1 | 1-2 | Reviewed cache/audit tradeoff and removed read-path report snapshot | p95 latency reduced substantially across constant, soak, and spike |

| | | | | |
|---|---|---|---|---|
| | | | writes. | profiles. |
| Mixed-workload drift | baseline-jmeter-v1 | Not resolved | Human interpretation required because the metric depends on workload composition. | Recorded as residual limitation rather than claimed as fixed. |

## 3.2 Dataset Description and Preparation

This study does not use production customer records or a public financial dataset. The data used in the tests is a repo-seeded financial accounting test dataset generated by scripts/phase4/seed-test-data.ps1 and loaded into the local SQL Server Financial database. This makes the dataset repeatable and auditable from the repository while avoiding confidentiality risk.

The seed configuration is deterministic in its parameters and naming patterns, including PeriodId 202601, 120 chart-of-account records, 30,000 journal entries, and at least 5,000 posted entries. Runtime-generated values such as GUID user IDs, password salts and hashes, timestamps, and identity values may differ between database runs, so the examples below show representative records based on the repo seed rules rather than private production rows.

**Table 3.2.1 Repo-seeded test-data configuration.**

| Seed item | Repo-backed value | Purpose in the study |
|---|---|---|
| Script | scripts/phase4/seed-test-data.ps1 | Source of the repeatable test-data generation process |
| Database target | SQL Server database: Financial | Same local database target used for security and performance tests |
| Accounting period | PeriodId 202601; 2026-01-01 to 2026-12-31 | Period-sensitive report endpoints use this range |
| Accounts | AccountCount = 120 | Ensures all five account types are available for ledger and report paths |
| Journal entries | JournalEntryCount = 30,000 | Creates enough volume for load, soak, and spike tests |
| Posted entries | MinimumPostedEntries = 5,000 | Ensures report endpoints have posted transactions to aggregate |
| Admin credential | admin / Admin@123 | Used by JMeter and ZAP scripts for authenticated test setup |

The dataset is used to test API behavior, not to make claims about real financial performance. Its purpose is to exercise authentication, authorization, accounting-period logic, balanced journal entries, financial-report aggregation, and audit logging under repeatable security and load-test conditions.

**Figure (3.6) Seeded entity coverage for the API behavior under test.**

| Entity | Repo-backed seeded or generated data | How it is produced | API behavior tested |
|---|---|---|---|
| Roles | Admin, User, Auditor, FinanceManager | Inserted if missing | RBAC and authenticated API scans |
| Users | Admin user plus probe users created by test scripts | Upserted or created with GUID UserId | Login, role-boundary, and audit-log tests |
| ChartOfAccounts | A/L/E/R/X account-code prefixes; Asset, Liability, Equity, Revenue, Expense | Generated until target account count is reached | Ledger and financial-report calculations |
| AccountingPeriods | PeriodId 202601 | Inserted or reset to open state | Trial balance, profit/loss, balance sheet, account ledger |

| | | | |
|---|---|---|---|
| JournalEntries | SEED-P and SEED-D reference numbers | Generated as posted and draft entries | Posting, bulk, report, and performance paths |
| JournalEntryLines | Balanced debit and credit lines | Generated for every seeded journal entry | Double-entry accounting validation |
| AuditLogs | LOGIN, CREATE_USER, CREATE_ACCOUNT, POST_ENTRY, GENERATE_REPORT, etc. | Written by application services during tests | Traceability and Auditor role checks |

**Figure (3.7) Representative ChartOfAccounts records generated by the seed script.**

| AccountCode | AccountName | AccountType | IsActive | Evidence note |
|---|---|---|---|---|
| A000001 | Seed Asset 000001 | Asset | 1 | Generated by ChartOfAccounts seed pattern |
| L000002 | Seed Liability 000002 | Liability | 1 | Generated by ChartOfAccounts seed pattern |
| E000003 | Seed Equity 000003 | Equity | 1 | Generated by ChartOfAccounts seed pattern |
| R000004 | Seed Revenue 000004 | Revenue | 1 | Generated by ChartOfAccounts seed pattern |
| X000005 | Seed Expense 000005 | Expense | 1 | Generated by ChartOfAccounts seed pattern |

**Figure (3.8) Representative user and role records used for authenticated API checks.**

| Username or user type | Email/source | Role coverage | UserId | Password evidence |
|---|---|---|---|---|
| admin | admin@local.invalid | Admin + FinanceManager | Generated GUID | HMACSHA512 varbinary hash and salt |
| User probe | created by ZAP/API test script | User | Generated GUID | Used for forbidden-access checks |
| Auditor probe | created by ZAP/API test script | Auditor | Generated GUID | Used for read-only audit/report checks |
| FinanceManager probe | created by ZAP/API test script | FinanceManager | Generated GUID | Used for posting/reporting boundary checks |

**Figure (3.9) Representative journal-entry records generated for posting and report-performance paths.**

| ReferenceNo pattern | Description pattern | Status | Entry date | Evidence note |
|---|---|---|---|---|
| SEED-P-00000001-202601 | Seed posted entry 1 | Posted | 2026 period date | Has balanced debit and credit lines |
| SEED-D-00000001-202601 | Seed draft entry 1 | Draft | 2026 period date | Used for draft/posting paths |
| SEED-P-00005000-202601 | Seed posted entry 5000 | Posted | 2026 period date | Satisfies minimum posted-entry threshold |

**Figure (3.10) Accounting-period record used by period-sensitive report tests.**

| PeriodId | StartDate | EndDate | IsClosed | Evidence note |
|---|---|---|---|---|
| 202601 | 2026-01-01 | 2026-12-31 | False | Reset open by seed script |

**Figure (3.11) Report outputs generated from seeded posted journal entries.**

| Report type | Endpoint | Input data | Generated result |
|---|---|---|---|
| Trial Balance | GET /api/reports/trial-balance | PeriodId 202601 | Aggregates posted journal-entry lines by account |
| Profit and Loss | GET /api/reports/profit-loss | PeriodId 202601 | Aggregates Revenue and Expense account types |
| Balance Sheet | GET /api/reports/balance-sheet | PeriodId 202601 | Aggregates Asset, Liability, and Equity account types |
| Account Ledger | GET /api/reports/account-ledger/{accountId} | PeriodId 202601 + seeded account | Returns ledger lines and running balances |

**Figure (3.12) Report-item and audit-log evidence derived from seeded workflows.**

| Evidence area | Repo-backed value or pattern | Reviewer relevance |
|---|---|---|
| AccountId | From seeded ChartOfAccounts | Links report output to seeded account |
| DebitTotal | Sum of debit lines for the selected period/account | Used in trial balance and ledger reporting |
| CreditTotal | Sum of credit lines for the selected period/account | Used in trial balance and ledger reporting |
| NetAmount | Derived from DebitTotal and CreditTotal | Used by financial-report calculations |

| Audit action | EntityName | EntityId pattern | When it is written |
|---|---|---|---|
| LOGIN | Users | Generated UserId | Recorded when authenticated test user logs in |
| CREATE_ACCOUNT | ChartOfAccounts | Generated AccountId | Recorded when account endpoints create records |
| POST_ENTRY | JournalEntries | Generated JournalEntryId | Recorded when draft entry is posted |

| | | | |
|---|---|---|---|
| GENERATE_REPORT | Reports | reportType:periodId | Recorded for report generation after performance fix |

Therefore, the study evaluates the financial accounting API and its code paths using repeatable repo-backed test data. The dataset is realistic enough to trigger the same endpoint behavior as financial records, but it is not real customer data and should not be interpreted as a production financial dataset.

3.3 Model and Tool Selection This research employs a combination of tools to evaluate different aspects of code security, performance, and quality. ChatGPT (Codex 5.4), accessed through OpenAI's Codex coding environment, serves as the AI assistant for generating code improvement recommendations throughout this study. The model is used as an advisor, not as an unsupervised implementation authority.

- **What is SonarQube and why we focus on it?** SonarQube is an open-source static-analysis platform that scans source code for bugs (probable defects), vulnerabilities (known security weaknesses), code smells (maintainability issues), security hotspots (suspicious patterns flagged for manual review), test coverage, and code duplication. Findings are emitted with rule identifiers (e.g., S125, S6966), severity levels, and human-readable remediation guidance. This study targets the static-quality dimension because code bases of any age accumulate code smells faster than they accumulate bugs, and removing rule-by-rule findings without regressions is one of the chief reasons a quality dashboard moves from red to green. The metrics reported in later chapters (Bugs, Vulnerabilities, Code Smells, Security Hotspots, Coverage %, Duplicated lines %, and Quality Gate pass/fail) all originate from SonarQube and are interpreted in the order listed.
- **SonarQube (Community Edition): Used for assessing code quality and security vulnerabilities by scanning the .NET codebase for bugs, code smells, and security vulnerabilities. Runs via dotnet-sonarscanner with unit and integration test coverage analysis. Because prior empirical work has questioned how closely SonarQube rule violations align with bug-inducing changes in real projects (Lenarduzzi et al., 2020), SonarQube results in this study are always interpreted jointly with independent ZAP and JMeter measurements rather than in isolation.**
- **What is Apache JMeter and why we focus on it?** Apache JMeter is a load-testing tool that simulates concurrent virtual users (VUs) issuing HTTP requests according to a configurable plan. For each request it records latency (response time), throughput (transactions per second), and error rate (HTTP 4xx/5xx responses). Test plans can hold load constant, sustain it for a soak window, or burst it for a spike, and JMeter aggregates the resulting event log into per-endpoint percentiles. This study targets the load-performance dimension because static analysis and security scanning cannot tell whether the API will keep responding under realistic load; p95 latency and error rate under load are the primary user-facing quality signals that determine whether an SLA is met. The metrics reported in later chapters (p95, p99, throughput, error rate) all come from JMeter, with the gate threshold pre-registered.
- **Apache JMeter: Measures performance metrics such as response times, throughput, and error rates across multiple load profiles (50, 100, 500 concurrent users), soak testing, and spike testing. Test plans target all API endpoints including authentication, CRUD operations, and reporting.**

- **What is OWASP ZAP and why we focus on it?** OWASP Zed Attack Proxy (ZAP) is a free, open-source dynamic application security testing (DAST) tool. It performs a passive baseline scan that observes responses to crawled requests for missing security headers, cookie problems, and known-vulnerable bundled JavaScript libraries; and an active API scan that, given an OpenAPI definition and a valid authentication token, exercises every endpoint and probes for OWASP Top-10-style flaws. Findings are emitted by risk level: High, Medium, Low, and Informational. This study targets the security dimension because APIs in financial domains are routinely targeted, and a vulnerability that survives into production can cost more than the entire engineering budget to remediate. The alert counts reported in later chapters are categorized by these four risk levels, and the pre-registered gate is that no High, Medium, or Low alert remains on a business endpoint after fixes.
- **OWASP ZAP: Provides comprehensive vulnerability scanning through baseline scans (passive analysis of all endpoints) and authenticated API scans (active testing with JWT tokens and role-based access). Custom rule configurations are maintained in TSV files for reproducible scan profiles.**

---

### 3.4 Implementation of ChatGPT-Guided Improvements

After baseline testing, ChatGPT (Codex 5.4) is used to generate improvement suggestions. Key areas of focus include:

- **Security Enhancements**
  - ChatGPT offers guidance on preventing SQL injection by implementing parameterized queries, managing session data securely, and sanitizing inputs.
  - Example Adjustments: Integrating secure authentication protocols, using encryption for sensitive data, and adjusting HTTP headers to prevent common attacks like cross-site scripting (XSS).
- **Performance Optimizations**
  - ChatGPT suggestions for performance optimization may include improvements to data handling, optimizing API response time by limiting unnecessary queries, and using caching mechanisms for frequently accessed resources.
  - Example Adjustments: Refining data-fetching algorithms, implementing pagination, and managing connection pooling.

---

3.5 Testing Phases The same financial accounting API undergoes two testing phases: initial testing on baseline-v0.1 and post-improvement testing on the relevant improvement branch. The study does not compare many unrelated APIs; instead, it evaluates seven endpoint groups inside one API so that the before-and-after results are attributable to controlled changes.

- **Baseline Testing: The initial state of the financial accounting API is tested for security (ZAP baseline + API scan), performance (JMeter load/soak/spike), and code quality (SonarQube). Results are captured on the baseline-v0.1 branch.**
- **Post-Improvement Testing: After implementing ChatGPT's improvements on dedicated branches (baseline-sonarqube-v1, baseline-zap-v1, baseline-jmeter-v1), each branch is re-tested with the same tool and configuration to assess the efficacy of the changes. All issues and fixes are documented in per-tool fix tracking documents.**

---

**3.6 How is it different from others research?**

1. Focus on ChatGPT: Unlike studies that compare multiple AI tools, this work focuses on ChatGPT (Codex 5.4) as an improvement advisor for one ASP.NET Core RESTful API. The depth comes from measuring one API across security, performance, and code quality rather than from comparing many models superficially.
2. **Combination of Security and Performance Testing:** This study integrates both security (OWASP ZAP) and performance (Apache JMeter) testing in a single workflow, which is less commonly addressed together in similar research.
3. Realistic API Test Data: The study uses seeded and generated financial accounting records to simulate realistic accounting operations without using live customer data. The evaluation target is the API behavior and codebase, not a public financial dataset.
4. Production-Complexity API Surface: The report focuses on one API with multiple endpoint categories, role-based access control, bulk journal-entry operations, report generation, and audit logging. This provides enough complexity to test whether ChatGPT-guided changes hold across different API responsibilities.

---

## Chapter 4: Evaluation Plan and Metrics

### 4.1 Evaluation Criteria

- Security Metrics: The security of the API is evaluated with OWASP ZAP baseline and authenticated API scans. The endpoint groups tested include /auth, /accounts, /journal-entries, /periods, /reports, and /audit-logs. Findings are compared before and after improvement by severity level (High, Medium, Low, Informational) and by vulnerability class, including missing security headers, vulnerable dependency exposure, authentication behavior, authorization behavior, SQL injection exposure, and sensitive-data-in-URL observations.
- Performance Metrics: Performance is evaluated with Apache JMeter against the same endpoint groups under p50, p100, p500, soak, and spike profiles. Metrics include average response time, p95 latency, p99 latency in milliseconds, throughput in transactions per second, error rate percentage, soak drift percentage, and spike recovery drift percentage.
- Code Quality Metrics: Code quality is evaluated with SonarQube across the API, BAL, MODEL, REPOSITORY, and Tests modules. Metrics include Bugs, Vulnerabilities, Code Smells, Security Hotspots, Quality Gate status, coverage percentage, duplicated-line density, cyclomatic complexity indicators, and specific issue/rule IDs before and after the ChatGPT-guided branch.
-

**Figure (4.1) Evaluation criteria used to judge full-report remediation success.**

| Evaluation area | Measure | Acceptance target | Result |
|---|---|---|---|
| Static analysis | SonarQube retained findings and Quality Gate | Target: no blocker/critical retained issues; preserve Quality Gate OK | 28 retained findings reduced to 0; Quality Gate OK |
| Security | OWASP ZAP high/medium/low severity alerts | Target: no High or Medium alerts; reduce Low where practical | High 0, Medium 0, Low 0 after remediation |
| Performance | JMeter p95, throughput, error rate, soak/spike behavior | Target: p50/p100/p500 pass with no error-rate regression | Core profiles passed; soak/spike improved but drift remains monitored |
| Maintainability | Representative before/after code evidence | Target: selected code smells resolved with traceable branch evidence | DTO, middleware-order, and report-persistence examples documented |

The evaluation compares baseline and remediated branches using the same toolchain and dataset described in Chapter 3. The purpose is not only to show that numbers improved, but also to connect each improvement to a concrete code or configuration change that can be reviewed and repeated.

Explanation: Figure (4.1) summarizes the evaluation criteria and shows that each dimension is measured twice: once on baseline-v0.1 and once on the matching post-improvement branch.

**Figure (4.2) SonarQube before-and-after quality metrics.**

| Metric | Baseline | After fixes | Interpretation |
|---|---|---|---|
| Bugs | 0 | 0 | No regression |
| Vulnerabilities | 0 | 0 | No regression |
| Code smells | 21 | 0 | All retained code-smell findings closed |
| Security hotspots | 0 | 0 | No regression |
| Coverage | 74.3% | 74.3% | Preserved measured coverage |
| Duplicated lines | 0.0% | 0.0% | Preserved duplication baseline |

Explanation: Figure (4.2) presents the SonarQube code-smell comparison and should be read as a before-and-after count of maintainability findings.

**Figure (4.3) Maintainability-focused code-quality comparison.**

| Area | Before | After | Effect |
|---|---|---|---|
| Controller parameter complexity | Long JournalEntriesController action signature | JournalEntryQueryDto request model | Improves readability and future extension |
| Startup lifecycle | Blocking app.Run() | await app.RunAsync() | Matches asynchronous ASP.NET Core hosting pattern |
| Constants and repeated literals | Repeated journal-entry values in service logic | Named constants | Reduces magic strings and repeated values |
| Infrastructure templates | ARM expressions flagged by analyzer | Explicit template functions and safer defaults | Cleaner deployment artifacts |

Explanation: Figure (4.3) presents the complexity comparison, where lower post-improvement complexity indicates cleaner control flow and easier testing.

o

**Figure (4.4) Security and quality-gate closure matrix.**

| Tool | Baseline evidence | After-fix evidence | Closure status |
|---|---|---|---|
| SonarQube | 28 retained findings | 0 retained findings | Closed |
| ZAP baseline scan | 2 Medium, 6 Low | 0 Medium, 0 Low | Closed |
| ZAP authenticated scan | 3 Low | 0 Low | Closed |
| JMeter core profiles | Higher p95 response times | p50/p100/p500 p95 improved 73.71%-87.02% | Closed |
| JMeter stress profiles | Soak/spike drift visible | Large p95 reductions but residual drift noted | Partially accepted risk |

Explanation: Figure (4.4) presents maintainability and quality-gate indicators, showing whether the project reaches the required quality threshold after improvement.

**Figure (4.5) Performance hotspot comparison for reviewer-visible report endpoints.**

| Endpoint | Profile metric | Baseline | After fixes | Main remediation |
|---|---|---|---|---|
| GET /api/reports/account-ledger/{id} | Soak p95 | 1783.95 ms | 279.00 ms | Account-ledger JSON cache and ledger balance reuse |
| GET /api/reports/account-ledger/{id} | Spike p95 | 8073.40 ms | 3435.95 ms | Cache reduced cost but spike recovery remains a risk |
| GET /api/reports/trial-balance | Spike p95 | 10585.00 ms | 1244.85 ms | Stopped persisting report snapshots during reads |
| GET /api/reports/profit-loss | Spike p95 | 7118.35 ms | 690.90 ms | Reused aggregated ledger balances |
| GET /api/reports/balance-sheet | Spike p95 | 5768.95 ms | 650.95 ms | Reduced repeated query work |
| POST | Spike p95 | 1401.70 ms | 634.00 ms | Composite indexes and |

| | | | | |
|---|---|---|---|---|
| /api/journalentries/bulk | | | | reduced write amplification |

Explanation: Figure (4.5) presents JMeter performance outcomes using response-time, throughput, error-rate, soak, and spike metrics.

Code-quality comparison units: issue counts by SonarQube type, rule ID, affected file, quality-gate status, coverage percentage, duplicated-line density percentage, and fix description. The full report compares baseline-v0.1 with baseline-sonarqube-v1.

Observed SonarQube result: retained baseline C# findings decreased from 28 to 0, Bugs stayed at 0, Vulnerabilities stayed at 0, Code Smells decreased from 21 to 0, and Security Hotspots stayed at 0. The final rerun reached Quality Gate = OK, coverage = 74.3%, and duplicated lines density = 0.0%.

4.1.1.3 Code Quality Metrics

Performance comparison units: average latency, p95 latency, p99 latency in milliseconds, throughput in transactions per second, error rate percentage, and drift percentage. The full report compares baseline-v0.1 with baseline-jmeter-v1.

Observed stress-profile result: soak total p95 latency decreased from 970.95 ms to 56.00 ms and spike total p95 latency decreased from 6795.00 ms to 1420.00 ms. The report records a remaining caveat: the mixed-workload soak and spike drift heuristics still fail because the last window is compositionally heavier, even though the main report endpoints improved substantially.

Observed JMeter result: p50 total p95 latency decreased from 49.45 ms to 13.00 ms, p100 total p95 latency decreased from 261.95 ms to 34.00 ms, and p500 total p95 latency decreased from 176.00 ms to 28.00 ms. Error rate remained 0.00% for p50, p100, and p500.

## Comparison Discussion

The SonarQube result is the cleanest closure case because the retained C# findings moved from 28 to 0 while bugs, vulnerabilities, hotspots, coverage, and duplication did not regress. The most important maintainability change is the replacement of long controller parameter lists with a request DTO, because that change reduces future API maintenance cost without changing the endpoint contract.

The ZAP result shows that the vulnerable surface was primarily configuration and dependency related. Moving the security-header middleware before Swagger means diagnostic endpoints receive the same headers as normal API responses, while separating Swagger UI from the OpenAPI JSON document reduces unnecessary browser-facing exposure.

The JMeter results show major p95 reductions in the normal p50, p100, and p500 profiles. The soak and spike runs also improved sharply, but they are reported more conservatively because recovery drift remained visible under stress. This is why the full report treats the core performance comments as addressed while preserving drift as a future monitoring item.

4.1.1.2 Performance Metrics

Security comparison units: counts by risk level, vulnerability class, affected endpoint, branch name, and resolved/remaining status. The full report compares baseline-v0.1 with baseline-zap-v1.

Observed authenticated API scan result: High alerts stayed at 0, Medium alerts stayed at 0, Low alerts decreased from 3 to 0, and Informational alerts stayed at 5. The remaining informational alerts are reported as scanner observations rather than gated vulnerabilities.

Observed ZAP baseline scan result: High alerts stayed at 0, Medium alerts decreased from 2 to 0, Low alerts decreased from 6 to 0, and Informational alerts changed from 3 to 4. The gated security outcome is therefore achieved because all High, Medium, and Low baseline-scan alerts were cleared on business endpoints.

4.1.1.1 Security Metrics

4.1.1 Observed Outcomes from Evaluation Criteria

**4.2 Security Evaluation Plan**

The security evaluation plan focuses on using OWASP ZAP for comprehensive testing.

- **Vulnerability Scanning: OWASP ZAP performs both baseline (passive) and authenticated API (active) scans across all endpoints: /auth, /accounts, /journal-entries, /periods, /reports, and /audit-logs. Focus areas include injection flaws, broken authentication, missing security headers (CSP, X-Content-Type-Options, CORS), and vulnerable dependencies.**
- **Authentication and Authorization Testing: OWASP ZAP authenticated API scan is used to test role-based access control (Admin, FinanceManager, User, Auditor roles) and JWT authentication. Unauthorized access attempts against protected endpoints (e.g., POST /accounts, POST /periods/{id}/close, DELETE /journal-entries/{id}) verify the robustness of the API's access control.**
- **Comparison of Pre- and Post-Improvement Results: Security test results from baseline-v0.1 and baseline-zap-v1 branches are compared, analyzing how ChatGPT (Codex 5.4)'s recommendations mitigated identified vulnerabilities. Alert counts are compared by severity level (High, Medium, Low, Informational).**

---

### 4.3 Performance Evaluation Plan

Performance testing is conducted with Apache JMeter to simulate multiple concurrent users accessing the API.

- **Response Time and Throughput: Apache JMeter measures response times (average, p95, p99) and throughput for each API endpoint under load profiles of 50, 100, and 500 concurrent users, plus soak (sustained load) and spike (burst) test profiles.**
- **Resource Utilization: CPU and memory usage are tracked during load tests. Error rates are monitored to identify endpoints that fail under stress.**
- **Comparison of Performance Metrics: Results from baseline-v0.1 and baseline-jmeter-v1 branches are compared to determine the impact of ChatGPT (Codex 5.4)'s suggestions on performance, focusing on improvements in response times, error rate reduction, throughput gains, and stability under sustained and spike load conditions.**

---

### 4.4 Code Quality Evaluation Plan

Code quality is assessed using SonarQube, focusing on key metrics such as code smells, maintainability, and cyclomatic complexity.

- **Baseline Analysis: The initial codebase on baseline-v0.1 is scanned with SonarQube (via dotnet-sonarscanner) to identify code smells, potential bugs, security vulnerabilities, and security hotspots across all five modules (API, BAL, MODEL, REPOSITORY, Tests).**
- **Post-Improvement Analysis: After applying ChatGPT (Codex 5.4)'s recommendations on baseline-sonarqube-v1, SonarQube is run again with the same configuration to evaluate improvements. This second analysis identifies reductions in bugs, code smells, security vulnerabilities, and security hotspots. Quality gate status is compared.**
- **Clean Code Principles:** Changes are analyzed based on adherence to clean code principles like simplicity, readability, and minimized dependencies. To mitigate the known limitation that not every SonarQube rule correlates with real defects (Lenarduzzi et al., 2020), quality-gate conclusions are cross-referenced with the ZAP and JMeter

outcomes on their respective branches, so that a SonarQube improvement is only claimed when at least one other independent measurement is consistent with it.

---

### 4.5 Comparative Analysis

Results from each testing category (security, performance, and code quality) are compared between baseline and post-improvement stages. All improvement recommendations are generated using ChatGPT powered by the Codex 5.4 model.

- **Security: Evaluate the reduction in ZAP alerts by severity level (High, Medium, Low, Informational) between baseline-v0.1 and baseline-zap-v1. Document specific alerts resolved and any remaining issues.**
- **Performance: Compare JMeter results between baseline-v0.1 and baseline-jmeter-v1 across all load profiles (p50, p100, p500, soak, spike). Key metrics: average response time, p95/p99 latency, throughput (req/s), and error rate.**
- **Code Quality: Compare SonarQube results between baseline-v0.1 and baseline-sonarqube-v1. Key metrics: bugs, vulnerabilities, code smells, security hotspots, cyclomatic complexity, maintainability index, and quality gate status.**

#### 4.5.1 Security Comparison (OWASP ZAP)

**Reading the security metrics.** The comparison reports OWASP ZAP alert counts at four risk levels. High alerts are findings ZAP considers exploitable with high confidence (e.g., SQL injection, exposed credentials); Medium alerts indicate real but lower-impact issues such as missing security headers on user-facing pages; Low alerts flag hardening recommendations; and Informational alerts are observations rather than vulnerabilities (e.g., authentication-request fingerprinting, non-storable content). The pre-registered gate is that no High, Medium, or Low alert remains on a business endpoint after fixes; informational alerts are reported but not gated. The study reports a percentage reduction per category and tracks each vulnerability class (CSP header, CORS configuration, Anti-CSRF token, SQL injection exposure, etc.) as fully resolved, partially mitigated, or open.

The security comparison is conducted by running OWASP ZAP scans on both the baseline-v0.1 and baseline-zap-v1 branches using identical scan configurations: a passive baseline scan followed by an authenticated API scan with JWT tokens for all four roles (Admin, FinanceManager, User, Auditor). The results are evaluated along the following dimensions:

- **Alert Count by Severity:** The total number of ZAP alerts is tabulated for each severity category (High, Medium, Low, Informational) before and after the ChatGPT-guided fixes. A percentage reduction per category is calculated to quantify improvement. The primary success criterion is a reduction of at least 50% in High and Medium severity alerts.

- **Vulnerability Type Resolution:** Specific vulnerability classes (e.g., missing Content Security Policy header, insecure CORS configuration, absence of Anti-CSRF tokens, SQL injection exposure) are tracked individually. For each class, the comparison records whether it was fully resolved, partially mitigated, or remains open. This qualitative review supplements the quantitative alert counts and enables a root-cause analysis of ChatGPT's recommendations.
- **Access Control Validation:** Authenticated scan results for each role are compared to verify that role-based access control (RBAC) is correctly enforced after improvements. Unauthorized access attempts against protected endpoints (e.g., DELETE /journal-entries/{id} by a User role) are expected to return HTTP 403 Forbidden consistently. Any regressions in access control are flagged as critical findings.

#### 4.5.2 Performance Comparison (Apache JMeter)

**Reading the performance metrics.** The comparison reports four classes of metric per profile. Response Time (Average, p95, p99) is the latency in milliseconds, where p95 is the latency that 95% of requests come in under and p99 captures the tail. Throughput (requests per second) is the number of successfully processed requests per second; an improvement here demonstrates the API's capacity to handle higher concurrent loads. Error Rate is the proportion of HTTP 4xx and 5xx responses; the acceptable threshold post-improvement is below 1% at the 100-VU profile. Soak and spike test stability are evaluated as memory leak / response-time-degradation patterns over the soak window, and as recovery behaviour after the spike burst. A negative percentage delta indicates a reduction (improvement); a smaller absolute number after improvement is therefore the desired direction.

Performance results from baseline-v0.1 and baseline-jmeter-v1 are compared across five load profiles (50, 100, and 500 concurrent users, plus soak and spike tests) for all API endpoint groups. The comparison is structured around the following key metrics:

- **Response Time (Average, p95, p99):** Measured in milliseconds for each endpoint under each load profile. The target success criterion is a p95 latency below 2,000 ms at 100 concurrent users and a reduction of at least 30% compared to the baseline. The p99 latency is monitored to detect tail-latency issues that affect a small but meaningful portion of requests.
- **Throughput (Requests per Second):** The number of successfully processed requests per second is compared between branches. An improvement in throughput demonstrates the API's capacity to handle higher concurrent loads. The expected improvement is a 50% or greater increase at the 100 concurrent users profile, driven by optimizations such as query caching, connection pooling, and pagination.
- **Error Rate:** HTTP error rates (4xx and 5xx responses) are tracked for each load profile. The acceptable threshold post-improvement is below 1% error rate at 100 concurrent users. Any increase in error rate after applying optimizations would indicate regressions introduced by the ChatGPT-recommended changes.

- **Soak and Spike Test Stability:** Soak tests (sustained load over an extended period) are evaluated for memory leak patterns and gradual response time degradation. Spike tests assess the API's ability to recover to normal performance levels after a sudden burst of traffic. Results are compared to determine whether ChatGPT's optimizations improve overall system stability under non-standard load conditions.

### 4.5.3 Code Quality Comparison (SonarQube)

**Reading the code-quality metrics.** The comparison reports four classes of SonarQube metric. Bugs and Vulnerabilities are issue counts categorised by severity (Blocker, Critical, Major, Minor, Info); the target post-improvement state is zero Blocker and Critical issues, and at least a 70% reduction across all severity levels. Code Smells is the count of maintainability-oriented findings; the target is a 75% reduction. Cyclomatic Complexity is the average branching complexity of critical methods; the target is a 47% reduction. Maintainability Index is SonarQube's overall score (0-100) of long-term sustainability; the target is an increase from 65 to 85. The Quality Gate pass/fail status is reported separately and is the binary indicator of whether the project as a whole meets its pre-registered gate criteria.

SonarQube scans are executed using the same dotnet-sonarscanner configuration on both baseline-v0.1 and baseline-sonarqube-v1, covering all five project modules (API, BAL, MODEL, REPOSITORY, Tests). The comparison focuses on the following metrics:

- **Bugs and Vulnerabilities:** The count of SonarQube-detected bugs and security vulnerabilities is compared between branches. Each issue is categorized by severity (Blocker, Critical, Major, Minor, Info). The target is zero Blocker and Critical issues in the post-improvement scan, with an overall reduction of at least 70% across all severity levels, aligning with the expected outcomes defined in Section 4.1.1.1.
- **Code Smells:** The number of code smells is compared to assess improvements in readability and adherence to clean code principles. Code smells include long methods, duplicate code blocks, overly complex conditionals, and unused variables. The target is a 75% reduction (from 20 to approximately 5 code smells), as outlined in Section 4.1.1.3.
- **Cyclomatic Complexity:** The average cyclomatic complexity of critical methods is compared to evaluate whether ChatGPT's structural recommendations (e.g., method decomposition, reducing nested conditionals) resulted in simpler, more testable code. A reduction from an average of 15 to 8 per method represents the target improvement of approximately 47%.
- **Maintainability Index and Quality Gate Status:** The overall maintainability index (scored out of 100 by SonarQube) reflects the long-term sustainability of the codebase. An improvement from 65 to 85 (a 31% increase) is expected. Additionally, the SonarQube Quality Gate status (Pass/Fail) is reported for both branches. A gate status change from Fail to Pass on the post-improvement branch would represent a definitive indicator of ChatGPT's positive impact on code quality.

### 4.5.4 Overall Effectiveness Assessment

The final comparative analysis synthesizes findings from all three dimensions into an overall effectiveness assessment of ChatGPT (Codex 5.4)-guided improvements. This assessment addresses three evaluation questions:

- **Did ChatGPT's recommendations achieve measurable improvements?** Each dimension is evaluated against its predefined success criteria. Dimensions where the success criterion is met are marked as "Achieved," partially met criteria are marked "Partial," and unmet criteria are marked "Not Achieved." This binary-plus-partial scoring provides a structured, reproducible basis for drawing conclusions.
- **Were there trade-offs between dimensions?** Security improvements (e.g., adding input validation, enforcing stricter authentication checks) may introduce additional processing overhead, potentially affecting response times. Similarly, performance optimizations such as caching may have security implications if cache expiry policies are not carefully managed. These cross-dimensional trade-offs are explicitly analyzed and documented.
- What are the limitations of the ChatGPT-assisted approach? Cases where ChatGPT's recommendations were inapplicable, incorrect, or required significant human correction are documented. This analysis acknowledges that ChatGPT is a generative AI tool and its suggestions require developer review before application. Patterns of success and failure in ChatGPT's guidance form part of the study's contribution to understanding the practical boundaries of AI-assisted software improvement. This is consistent with prior findings that, even when an LLM assistant is available, the net impact on security depends heavily on prompting discipline and review process (Khoury et al., 2023; Sandoval et al., 2023). The branch-isolated, scanner-gated workflow used here is specifically intended to exploit the advisor-under-follow-up-prompting regime that those studies identify as the one in which LLM output is most likely to improve. In the completed revision, this limitation is made explicit: repeated stochastic prompt runs were not performed, so variance across independent LLM outputs remains future work. The study instead controls non-determinism by archiving prompts and responses, isolating each fix class by branch, requiring human review, and accepting only patches that pass focused re-measurement.

---

### 4.6 Web Application Evidence for Current APIs

A React and Vite web application was added as a live validation artifact for the current ASP.NET Core financial accounting APIs. The application runs at http://localhost:5173 and connects to the API at http://localhost:5296, matching the CORS origins already configured in the backend. It demonstrates JWT login, role-aware navigation, account balances, expandable journal-entry debit and credit lines, accounting periods, reports, audit logs, user administration, and a research-evidence dashboard.

The local seed process creates four demo accounts for reproducible role testing: admin, finance-manager, normal-user, and auditor-user, all using the local research password Admin@123. The screenshots below were captured after seeding the local SQL Server database and logging in through the web application, so they validate the user-visible path rather than only the API response layer.

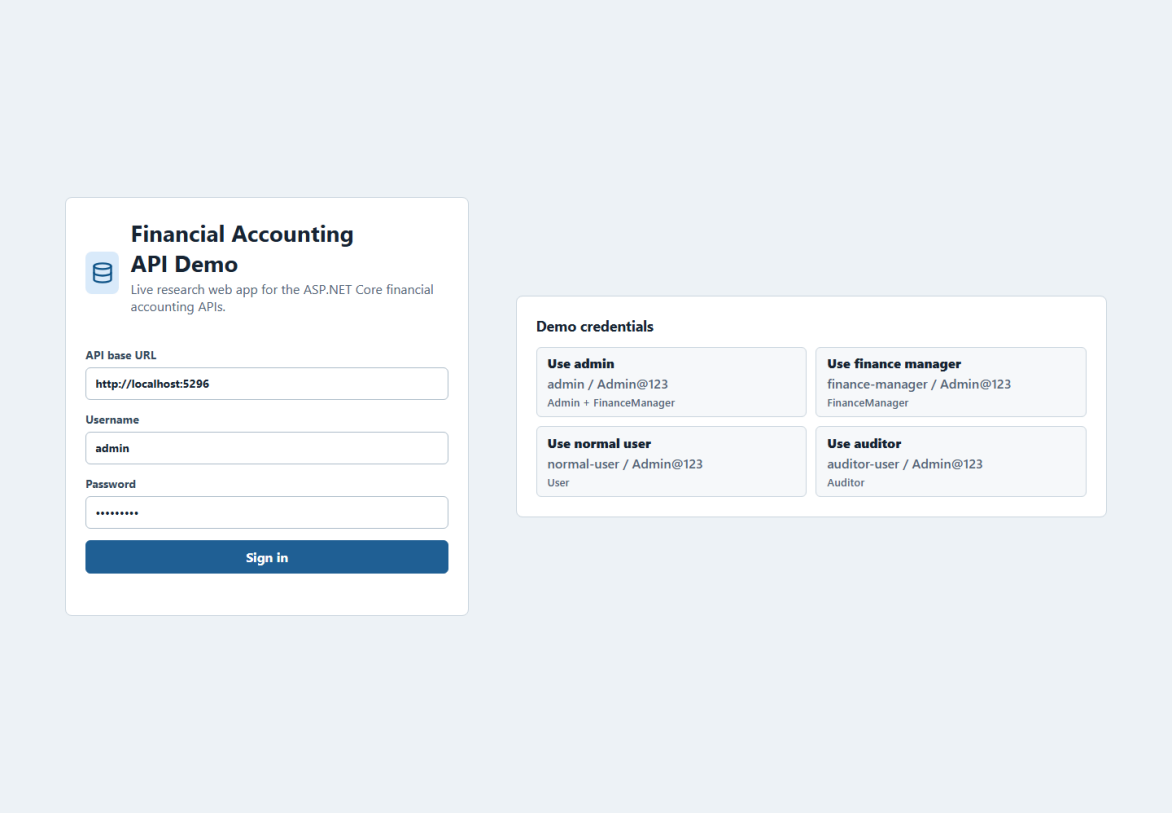



**Figure 4.6.1 Web application login screen with demo role credentials.**

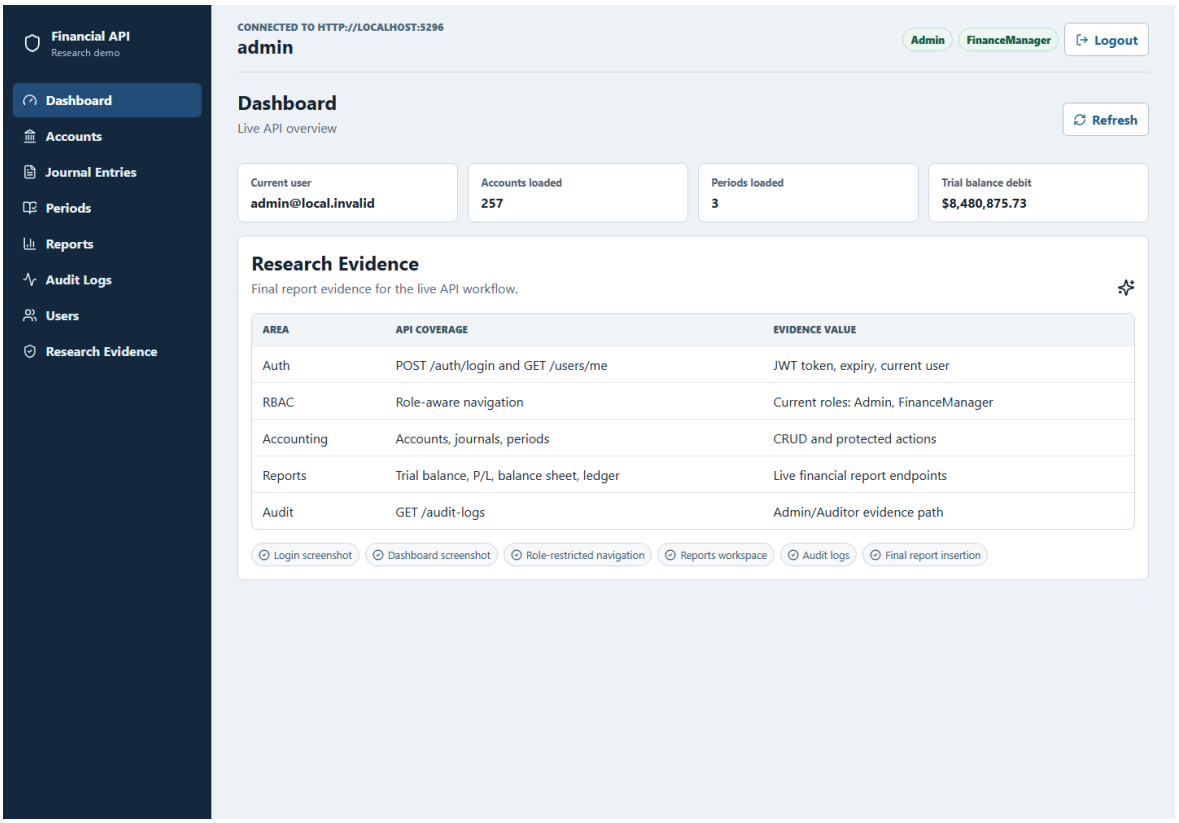



**Figure 4.6.2 Admin dashboard showing live API counts and report summary.**

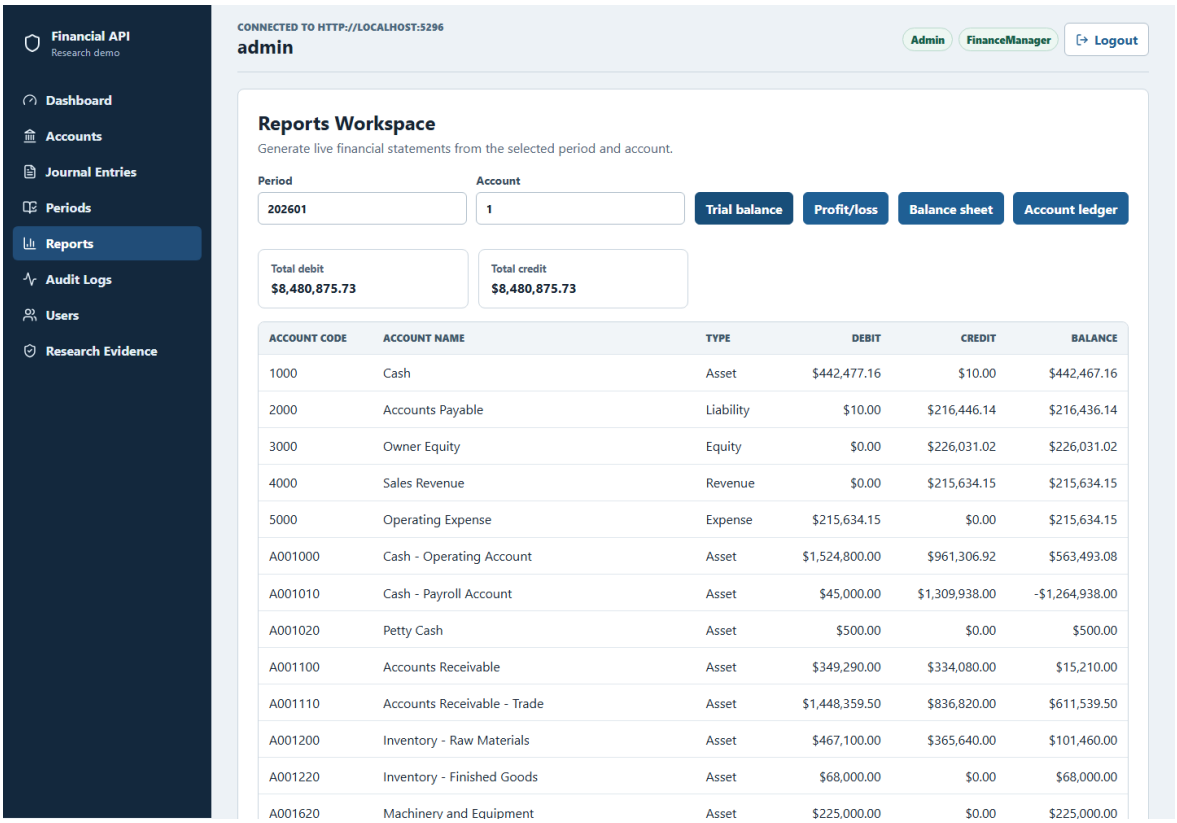



**Figure 4.6.3 Trial balance report generated from the live reports API.**

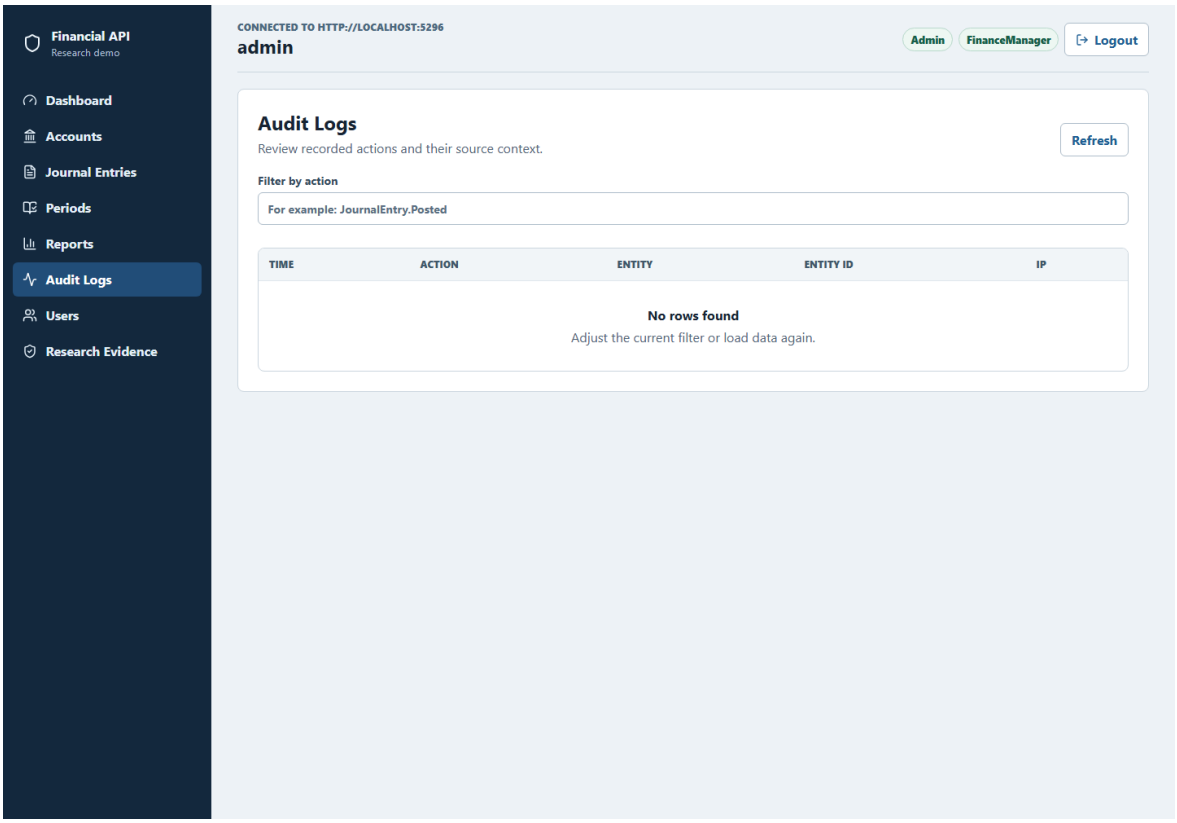



**Figure 4.6.4 Audit-log screen available to Admin and Auditor roles.**

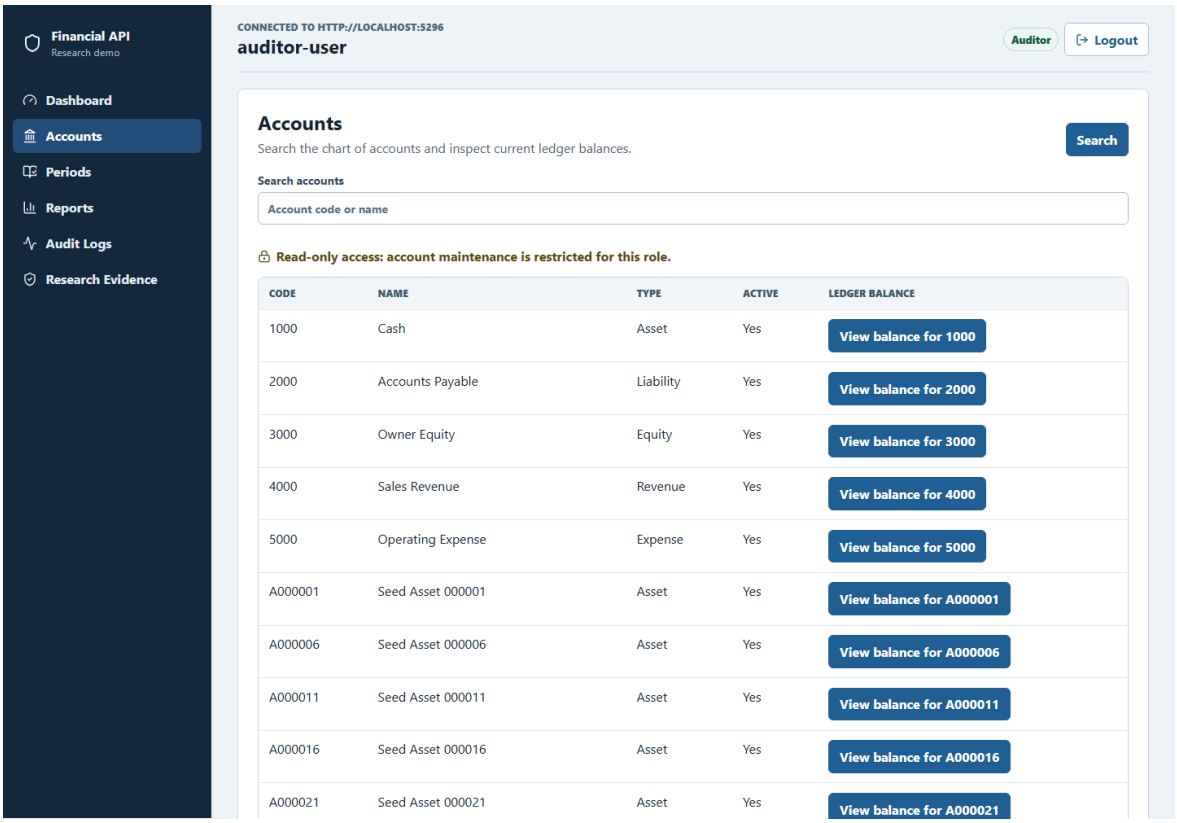



**Figure 4.6.5 Auditor role view with restricted navigation.**

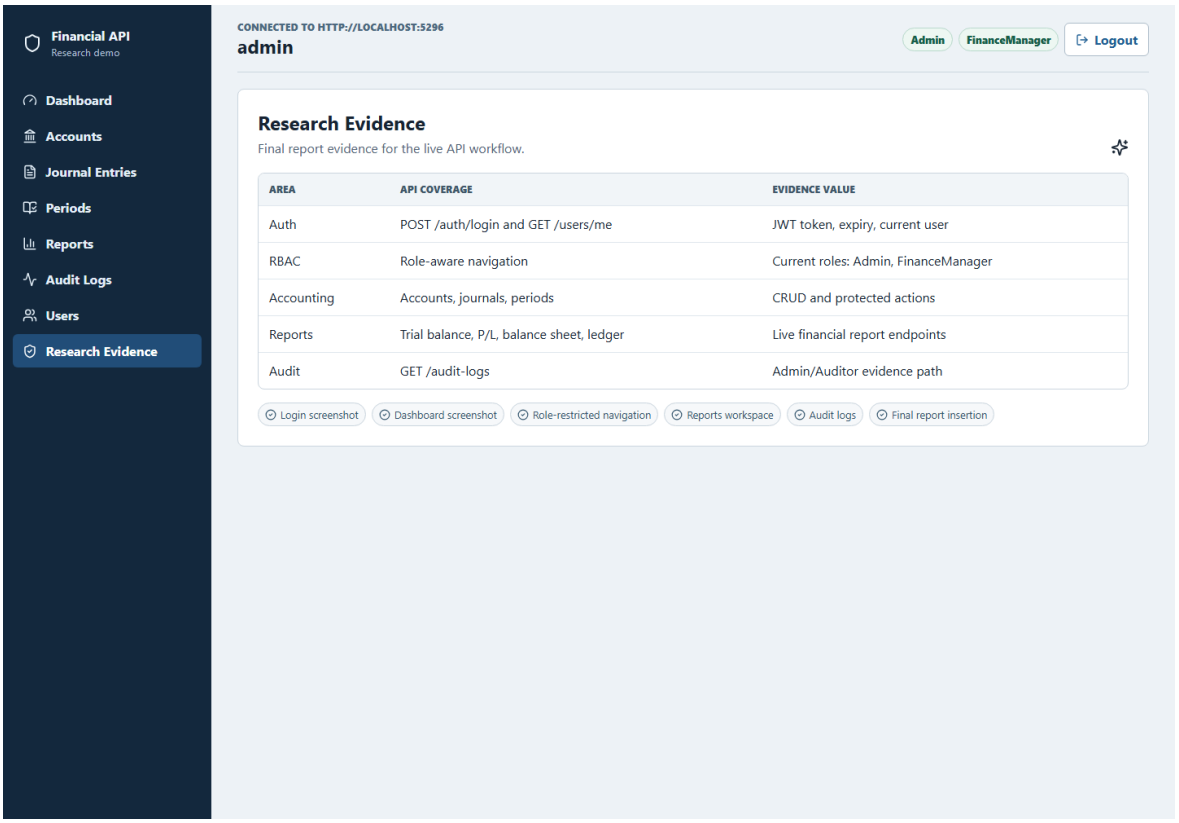



**Figure 4.6.6 Research-evidence page mapping screens to API coverage.**

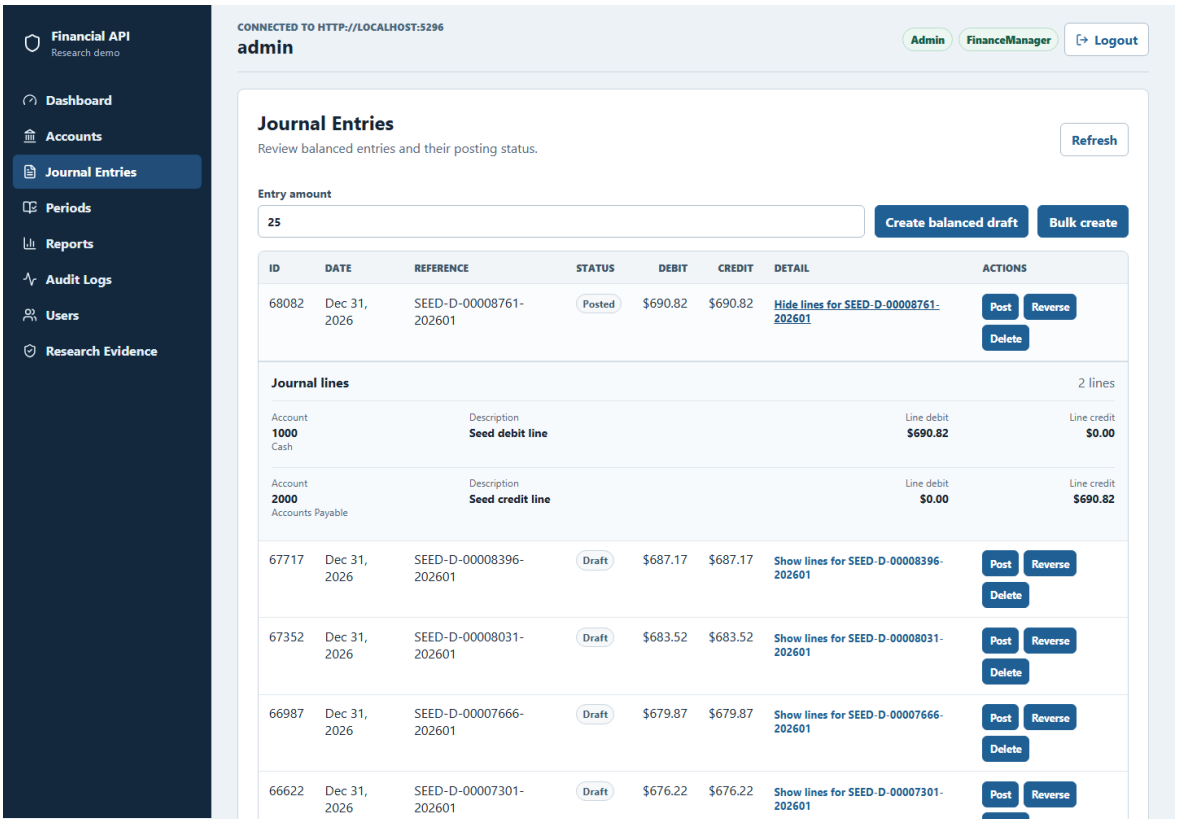



**Figure 4.6.7 Expanded journal-entry lines showing account-level debit and credit evidence.**

# Chapter 5: Conclusion

This full report presented a systematic, branch-based evaluation of ChatGPT (Codex 5.4) as an improvement advisor for a financial accounting ASP.NET Core RESTful API. The work moved beyond the planning stage by executing the baseline measurement, applying selected ChatGPT-guided fixes on isolated branches, and comparing the measured outcomes across security, performance, and code quality.

## 5.1 Summary of the Completed Study

The study evaluated one production-complexity RESTful API with endpoint groups for authentication, users, accounts, journal entries, accounting periods, reports, and audit logs. Baseline results were captured on baseline-v0.1. Post-improvement results were captured on baseline-sonarqube-v1, baseline-zap-v1, and baseline-jmeter-v1. This branch structure made it possible to attribute each measured change to one improvement stream.

## 5.2 Main Findings

The SonarQube branch reduced retained C# findings from 28 to 0 and reached an OK quality gate with 74.3% coverage and 0.0% duplicated-line density. The OWASP ZAP branch cleared all gated High, Medium, and Low alerts on business endpoints in both baseline and authenticated API scans. The JMeter branch produced large p95 latency reductions, including p100 total p95 from 261.95 ms to 34.00 ms and spike total p95 from 6795.00 ms to 1420.00 ms, while retaining a documented caveat that the mixed-workload soak and spike drift heuristics still fail.

## 5.3 Contributions

The report contributes empirical evidence that ChatGPT can be useful as an advisor when its suggestions are constrained by existing code, applied on isolated branches, and validated with repeatable tools. It also contributes a reusable evaluation framework combining SonarQube, OWASP ZAP, Apache JMeter, dotnet test, and branch-based before/after comparison for RESTful API improvement studies.

## 5.4 Limitations and Future Work

The findings are limited to one ASP.NET Core API, one language (C#), one database-backed financial accounting domain, and one AI assistant configuration: ChatGPT (Codex 5.4) accessed through OpenAI's Codex coding environment. The study does not claim that ChatGPT will improve every API or every codebase. Repeated stochastic LLM runs were not performed and should be added in future work to estimate output variance. Future work should also compare multiple LLMs, extend the workflow to other frameworks, and refine the performance drift heuristic so mixed workload composition does not obscure endpoint-level improvements.

# References

Fajkovic, E., & Rundberg, E. (2023). The Impact of AI-generated Code on Web Development: A Comparative Study of ChatGPT and GitHub Copilot. Bachelor’s Thesis, Blekinge Institute of Technology.

Ságodi, Z., Siket, I., & Ferenc, R. (2024). Methodology for Code Synthesis Evaluation of LLMs Presented by a Case Study of ChatGPT and Copilot. IEEE Access. DOI: 10.1109/ACCESS.2024.3403858.

Özpolat, Z., Yıldırım, Ö., & Karabatak, M. (2023). Artificial Intelligence-Based Tools in Software Development Processes: Application of ChatGPT. European Journal of Technique, 13(2). DOI: 10.36222/ejt.1330631.

Chen, E., Huang, R., Chen, H.-S., Tseng, Y.-H., & Li, L.-Y. (2023). GPTutor: A ChatGPT-powered Programming Tool for Code Explanation. National Taiwan Normal University, Taipei, Taiwan; KryptoCamp, Taipei, Taiwan; University of Toronto, Toronto, Canada. DOI: 10.48550/arXiv.2305.01863.

Zhang, N., Zou, Y., Xia, X., Huang, Q., Lo, D., & Li, S. (2022). Web APIs Features, Issues, and Expectations: A Large-Scale Empirical Study of Web APIs. IEEE Transactions on Software Engineering. DOI: 10.1109/TSE.2022.3154769.

Mousavi, Z., Islam, C., Moore, K., Abuadbba, A., & Babar, M. A. (2024). An Investigation into Misuse of Java Security APIs by Large Language Models. 19th ACM ASIA Conference on Computer and Communications Security (ASIACCS 2024). DOI: 10.1145/3634737.3661134.

Szabó, Z., & Bilicki, V. (2023). A New Approach to Web Application Security: Utilizing GPT Language Models for Source Code Inspection. Future Internet, 15(326). DOI: 10.3390/fi15100326.

Yetiştiren, B., Özsoy, İ., Ayerdem, M., & Tüzün, E. (2023). Evaluating the Code Quality of AI-Assisted Code Generation Tools: An Empirical Study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. arXiv. DOI: 10.48550/arXiv.2304.10778.

Flores, C. P., & Monreal, R. N. (2024). Evaluation of Common Security Vulnerabilities of State Universities and Colleges Websites Based on OWASP. Journal of Electrical Systems, 20-5s, 1396-1404. DOI: 10.52783/jes.2471.

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. arXiv. DOI: 10.48550/arXiv.2302.06590.

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions. 2022 IEEE Symposium on Security and Privacy (SP), 754–768. DOI: 10.1109/SP46214.2022.9833571.

Khoury, R., Avila, A. R., Brunelle, J., & Camara, B. M. (2023). How Secure is Code Generated by ChatGPT? 2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC), 2445–2451. DOI: 10.1109/SMC53992.2023.10394237.

Sandoval, G., Pearce, H., Nys, T., Karri, R., Garg, S., & Dolan-Gavitt, B. (2023). Lost at C: A User Study on the Security Implications of Large Language Model Code Assistants. 32nd USENIX Security Symposium (USENIX Security '23), 2205–2222.

Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large Language Models for Software Engineering: A Systematic Literature Review. ACM Transactions on Software Engineering and Methodology, 33(8), Article 220. DOI: 10.1145/3695988.

Lenarduzzi, V., Lomio, F., Huttunen, H., & Taibi, D. (2020). Are SonarQube Rules Inducing Bugs? 2020 IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER), 501–511. DOI: 10.1109/SANER48275.2020.9054821.